BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI TP. HỒ CHÍ MINH
KHOA KỸ THUẬT ĐIỆN TỬ 2

BÁO CÁO ĐỒ ÁN HỌC PHẦN

CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

Ngành Kỹ thuật Điện - Điện tử

ĐỀ TÀI

TÌM ĐỒNG THỜI GIÁ TRỊ NHỎ NHẤT VÀ LỚN NHẤT

Mã đề DT046

Giảng viên hướng dẫn	Hồ Nhựt Minh
Sinh viên thực hiện	Nguyễn Hoàng Nam
Mã số sinh viên	N24DCDT051
Lớp	D24CQDT01-N

THÀNH PHỐ HỒ CHÍ MINH THÁNG 10 NĂM 2026

BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI TP. HỒ CHÍ MINH
KHOA KỸ THUẬT ĐIỆN TỬ 2

BÁO CÁO ĐỒ ÁN HỌC PHẦN

CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

Ngành Kỹ thuật Điện - Điện tử

ĐỀ TÀI

TÌM ĐỒNG THỜI GIÁ TRỊ NHỎ NHẤT VÀ LỚN NHẤT

Mã đề DT046

Giảng viên hướng dẫn	Hồ Nhựt Minh
Sinh viên thực hiện	Nguyễn Hoàng Nam
Mã số sinh viên	N24DCDT051
Lớp	D24CQDT01-N
Học phần và nhóm	INT13185 Nhóm 03
Phiên bản báo cáo	31/10/2026 (Tiến độ 2 - Commit 43bc4a3)

THÀNH PHỐ HỒ CHÍ MINH THÁNG 10 NĂM 2026

PHIẾU GIAO ĐỀ TÀI

Học phần INT13185 Nhóm 03 Giảng viên Hồ Nhựt Minh

Thông tin	Sinh viên điền theo bảng phân công
Họ tên MSSV lớp	Nguyễn Hoàng Nam N24DCDT051 D24CQDT01-N
Mã đề và tên đề	DT046 Tìm đồng thời giá trị nhỏ nhất và lớn nhất
Căn cứ giáo trình	Chương 4, 4.1.3 Tìm đồng thời giá trị nhỏ nhất và lớn nhất; Bài tập chương 4, câu 18; trang in 91–92, 111; trang PDF 103–104, 123
Ngày giao ngày nộp	Ngày giao: 17/10/2026 — Hạn nộp Mốc 2: 01/11/2026

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG - CƠ SỞ TẠI TP. HỒ CHÍ MINH

PHIẾU GIAO ĐỀ TÀI: DT046 - INT13185 - Nhóm 03 - Giảng viên: Hồ Nhựt Minh

Sinh viên: Nguyễn Hoàng Nam - MSSV: N24DCDT051 - Lớp: D24CQDT01-N

1. Tên đề tài: Tìm đồng thời giá trị nhỏ nhất và lớn nhất

2. Căn cứ giáo trình: Chương 4, 4.1.3 Tìm đồng thời giá trị nhỏ nhất và lớn nhất; Bài tập chương 4, câu 18; trang in 91–92, 111; trang PDF 103–104, 123.

3. Đề bài: Viết chương trình tìm cả số nhỏ nhất và lớn nhất bằng cách chia mảng thành các đoạn. Sinh viên xử lý riêng đoạn một hoặc hai phần tử, rồi ghép hai cặp kết quả từ hai nửa. Bài làm chú trọng đếm phép so sánh và giải thích vì sao xử lý hai phần tử bằng một lần so sánh có lợi.

4. Dữ liệu vào: Nhập hoặc khai báo mảng a gồm 1 đến 1024 số nguyên từ -1000 đến 1000. Mảng không rỗng. Hàm làm việc trên đoạn chỉ số l đến r, tính cả hai đầu; được phép nhận mảng có giá trị trùng.

5. Kết quả ra: In min, max và số phép so sánh giá trị do hàm chia để trị thực hiện. Việc kiểm điều kiện l = r không tính vào biến so sánh phần tử.

6. Ví dụ 1: a = [-3, 7, 2, 9] -> min = -3; max = 9; so_sanh = 4. (Hai cặp dùng mỗi cặp một so sánh, ghép hai kết quả dùng thêm hai).

7. Ví dụ 2: a = [5] -> min = 5; max = 5; so_sanh = 0. (Một phần tử đồng thời là nhỏ nhất và lớn nhất, không cần so giá trị).

8. Năm bước thực hiện:

- Bước 1: Quy định ý nghĩa min_max(a, l, r), kết quả trả về một cặp và số so sánh phát sinh.
- Bước 2: Với một phần tử, trả chính nó hai lần; với hai phần tử, chỉ so một lần để trả cả min, max.
- Bước 3: Chia đoạn dài hơn thành hai nửa, tính cặp min, max của từng nửa bằng lời gọi đệ quy.
- Bước 4: Ghép bằng một so sánh giữa hai min và một so sánh giữa hai max, tăng đúng hai lần đếm.
- Bước 5: Đối chiếu với duyệt trực tiếp trên các mảng nhỏ, vẽ cây cho n=4 và kiểm số so sánh là 4.

9. Các hàm nên viết: min_max(a, l, r), min_max_truc_tiep(a), dem_so_sanh(a).

10. Năm ca cần kiểm tra:
- Ca 1: a = [8, 2] -> min = 2, max = 8, chỉ 1 phép so sánh giá trị.
- Ca 2: a = [4, 4] -> min = max = 4, vẫn chỉ 1 phép so sánh.
- Ca 3: a = [-5, -1, -3] -> min = -5, max = -1, không khởi tạo max bằng 0.
- Ca 4: a = [1, 2, 3, 4, 5, 6, 7, 8] -> min = 1, max = 8, bản chia đôi dùng 10 so sánh.
- Ca 5: a = [9, 1, 7, 3, 5] -> min = 1, max = 9; kiểm riêng n lẻ, không ép công thức cho n chẵn.

11. Phân tích và nhận xét: Thời gian O(n), độ sâu O(log n), không cần sao chép mảng con. Khi n là lũy thừa 2 và cơ sở hai phần tử dùng một so sánh, có T(2)=1, T(n)=2T(n/2)+2, suy ra 3n/2 - 2. Với n bất kỳ, tính đúng số theo các đoạn thật; không áp công thức chính xác một cách máy móc.

Yêu cầu bổ sung chung: ≥12 ca kiểm độc lập; 11 ảnh thực thi/kiểm thử/clone/GitHub; bảng truy vết; lập luận tính đúng; ba quy mô hoặc cấu hình hợp lệ; URL thật, SHA mã được kiểm và permalink. Nội dung thuật toán và miền dữ liệu vẫn theo phiếu cá nhân.

Sinh viên thực hiện	**Giảng viên hướng dẫn**
(Ký và ghi rõ họ tên)	(Ký và ghi rõ họ tên) .....

PHIẾU CHẤM VÀ ĐỐI CHIẾU MINH CHỨNG

Dùng rubric chung và hướng dẫn minh chứng đúng mã đề trong tệp 00_HUONG_DAN_SINH_VIEN.html. Không tự điền điểm khi chưa được chấm.

Tiêu chí	Trọng số	GV1 /10	GV2 /10	Quy đổi
Bài toán và đầu vào đầu ra	1.5	.....	.....	.....
Biểu diễn dữ liệu và phần tự cài	2	.....	.....	.....
Thuật toán và truy vết	2.5	.....	.....	.....
Kiểm thử và độ phức tạp	2	.....	.....	.....
Báo cáo minh chứng GitHub	1	.....	.....	.....
Bảo vệ và giải thích	1	.....	.....	.....

Quy đổi từng tiêu chí = điểm /10 × trọng số. Hai người chấm lấy trung bình điểm; một người chấm ghi rõ. Tổng điểm ...../10. Nhận xét .....

Người chấm 1 (ký, họ tên) ..... Người chấm 2 .....

CÁCH ĐỐI CHIẾU RUBRIC

Ba trạng thái Đầy đủ / Một phần / Chưa có dưới đây giúp đối chiếu nội dung và bằng chứng. Đây không phải ba mức điểm thay rubric hiện hành. Sinh viên điền vị trí bằng chứng; giảng viên xác định điểm sau khi kiểm báo cáo, mã, chạy lại và bảo vệ. Giữ cột điểm trống trước khi chấm.

Mọi tiêu chí áp dụng theo phiếu cá nhân: đề dùng biến/mảng không phải bổ sung cây, đồ thị hoặc lớp; chỉ làm nhiều thuật toán/đối chứng/cấu trúc tự cài khi phiếu yêu cầu.

C1 Đặc tả bài toán, đầu vào và đầu ra — trọng số 1.5

Đầy đủ: Đúng phiếu; miền, chỉ số và output rõ; hai ví dụ kiểm được; rỗng/trùng/hòa/vô nghiệm/input sai được nêu khi có áp dụng. Một phần: Nêu đúng bài toán chính nhưng còn thiếu hoặc chưa nhất quán một số miền, quy ước hay ví dụ. Chưa có: Chưa có đặc tả hoặc chưa có ví dụ đủ dữ liệu để đối chiếu bài toán được giao.

C2 Biểu diễn dữ liệu và phần tự cài — trọng số 2

Đầy đủ: Biểu diễn phù hợp; thao tác bắt buộc tự cài đủ; chỉ số/liên kết/trạng thái và bất biến đúng; công cụ có sẵn dùng trong phạm vi phiếu. Một phần: Biểu diễn chính dùng được nhưng còn thiếu thao tác, kiểm tra, bất biến hoặc giải thích phần tự cài. Chưa có: Chưa có biểu diễn hoạt động được hoặc chưa xác định được phần tự cài cần kiểm.

C3 Tính đúng của thuật toán và lần vết — trọng số 2.5

Đầy đủ: Thuật toán đáp ứng phiếu; lần vết khớp mã và input; giải thích được dừng, bất biến/truy hồi; kiểm nghiệm hợp lệ, tối ưu hoặc đầy đủ đúng yêu cầu. Một phần: Có thuật toán và lần vết cơ bản nhưng còn thiếu nhánh, trạng thái hoặc mắt xích lập luận; nêu rõ phần chưa kiểm được. Chưa có: Chưa có mã/lần vết đủ đối chiếu hoặc chưa có lập luận gắn thuật toán với kết quả yêu cầu.

C4 Kiểm thử, đối chiếu và độ phức tạp — trọng số 2

Đầy đủ: Đủ ca độc lập, ví dụ và ca phiếu; expected có cơ sở; actual/kết luận chạy lại được; đếm/đo và thời gian/bộ nhớ đúng bản cài, tách các pha cần thiết. Một phần: Có kiểm thử và phân tích cơ bản nhưng còn thiếu nhóm ca, cách xác định kỳ vọng, quy ước bộ đếm hoặc một phần khảo sát. Chưa có: Chưa có bảng kiểm thử đối chiếu được hoặc chưa có phân tích chi phí gắn với mã.

C5 Báo cáo, minh chứng và GitHub — trọng số 1

Đầy đủ: Đúng khung và thông tin thật; hình/bảng có dẫn giải; đủ 11 ảnh đọc được; repo/README/lệnh chạy truy cập được; phiên bản báo cáo, mã, ảnh và hồ sơ nộp đối chiếu được. Một phần: Có báo cáo và mã nhưng còn thiếu hoặc chưa rõ một số mục, ảnh, chú thích, nguồn, lệnh hay bằng chứng phiên bản. Chưa có: Chưa có báo cáo hoặc chưa có bộ minh chứng đủ để xác minh sản phẩm và phiên bản.

C6 Bảo vệ và giải thích sản phẩm — trọng số 1

Đầy đủ: Giải thích rõ bài toán, hàm, trạng thái và chi phí; dự đoán, chạy và phân tích input mới tại đúng phiên bản; nêu giới hạn sản phẩm. Một phần: Giải thích được ý tưởng/kết quả cơ bản nhưng cần gợi ý khi lần vết, giải thích mã hoặc xử lý input mới. Chưa có: Chưa có bằng chứng bảo vệ hoặc chưa giải thích được bài toán, mã và kết quả của mình.

Mỗi tiêu chí chấm 0–10. Hai người chấm: lấy trung bình tiêu chí; một người: dùng điểm người đó và ghi rõ. Điểm quy đổi = điểm tiêu chí × trọng số / 10. Cộng sáu dòng rồi làm tròn một chữ số thập phân; không làm tròn từng dòng trước khi cộng. Không tự đặt trần điểm toàn bài hoặc công thức phạt mới; ghi rõ bằng chứng còn thiếu và chấm phần kiểm được.

HƯỚNG DẪN ĐIỀN KHUNG BÁO CÁO

Đây là khung trống dùng chung cho mọi đề DT001–DT091. Điền theo phiếu cá nhân; các ô kết quả, ảnh, số liệu, URL và điểm chưa được điền. Không cần thêm bài toán hoặc thuật toán ngoài phiếu.

Bấm vào phần trong ngoặc vuông để thay bằng nội dung của mình. Các ô có thể gõ nhiều dòng; không có khóa chỉnh sửa hay macro. Với khung ảnh, chọn chữ bên trong rồi Chèn → Hình ảnh; đặt ảnh Cùng dòng với văn bản. Khi cần thêm ca/hàm, sao chép dòng tương ứng.

Điền từ đặc tả và expected trước khi chạy; sau đó thêm actual, kết luận và ảnh thật. Xóa câu hướng dẫn/chỗ không áp dụng có giải thích; giữ rubric để giảng viên chấm. Điền cả hai bìa, cập nhật toàn bộ mục lục/danh mục và kiểm PDF cuối trước khi nộp.

Thay mọi phần trong ngoặc vuông bằng nội dung đã thực hiện. Các câu gợi ý chỉ để hướng dẫn, phải xóa hoặc viết lại trước khi nộp. Không giữ các chỗ trống như một kết quả hoàn thành.

Giữ cấu trúc theo mẫu môn: hai bìa, phiếu, rubric, cam đoan, cảm ơn nếu dùng, tóm tắt, mục lục, danh mục, năm chương, tham khảo và phụ lục. Đây là khung theo hình thức đồ án dùng cho học phần.

Nội dung thân bài dùng style Nội dung: A4; lề trái 3 cm, các lề khác 2 cm; Times New Roman 13; giãn dòng 1,5. Chương đầu trang, hình có chú thích dưới, bảng có tên trên; số theo chương.

Ảnh và bảng phải được nhắc và giải thích trong nội dung. Chèn ảnh In Line with Text, đọc được trên bản in, không kéo méo. Hình lý thuyết không thay ảnh thực thi.

Mở 00_HUONG_DAN_SINH_VIEN.html và rubric chung trước khi viết; tra mã đề để chọn bất biến, kết quả và ca biên phù hợp. Không thêm thuật toán thứ hai nếu phiếu không yêu cầu.

LỜI CAM ĐOAN

Em xin cam đoan bản báo cáo tiến độ 2 này là công trình học tập do chính bản thân em tự thực hiện dưới sự hướng dẫn của giảng viên Hồ Nhựt Minh. Toàn bộ mã nguồn, các ví dụ minh họa, bộ kiểm thử 12 ca độc lập, các số liệu thực nghiệm đo đạc hiệu năng và 11 hình ảnh chụp màn hình minh chứng đều được thực thi trung thực trên môi trường Julia 1.12.7, Windows PowerShell và kho mã nguồn GitHub thật. Em hoàn toàn chịu trách nhiệm về tính trung thực của các thông tin và mã nguồn trong báo cáo.

TP. Hồ Chí Minh ngày ..... tháng ..... năm .....

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

[HỌ VÀ TÊN]

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành tới thầy Hồ Nhựt Minh đã tận tình hướng dẫn, truyền đạt kiến thức và định hướng phương pháp học tập nghiên cứu cấu trúc dữ liệu và giải thuật một cách chuẩn mực và thực chất.

TÓM TẮT

Báo cáo tiến độ 2 (Mốc 2) hoàn thành toàn diện các mục tiêu trọng tâm của đề tài DT046: Tìm đồng thời giá trị nhỏ nhất và lớn nhất. Về mặt lý thuyết, đồ án đã phân tích chi tiết nguyên lý chia để trị, cơ chế tối ưu tại trường hợp cơ sở 2 phần tử (chỉ tốn đúng 1 phép so sánh giá trị) và bước ghép 2 nửa (tốn 2 phép so sánh), thiết lập và giải hệ thức đệ quy T(n) = 2T(n/2) + 2 để chứng minh số phép so sánh đạt 3n/2 - 2 khi n là lũy thừa 2, tiết kiệm khoảng 25% số phép so sánh so với phương pháp duyệt tuần tự 2n - 2. Về mặt thực nghiệm và mã nguồn, đồ án đã hoàn thiện trọn vẹn 3 hàm cơ sở trong module DeTai (Julia 1.12.7): kiem_tra_dau_vao, min_max và min_max_truc_tiep, xây dựng kịch bản benchmark.jl đo đạc trên 4 quy mô n in {5, 20, 100, 500}, hoàn thành trọn vẹn bộ kiểm thử 12 ca độc lập (45 phép assert) đạt 100% Pass trong test/runtests.jl. Mã nguồn đã được chốt mốc Commit C (43bc4a301fafb9d54c9396c20bfcfe67fded5a1e) và đồng bộ lên GitHub. Báo cáo hoàn thành đầy đủ Chương 3, Chương 4, bảng lần vết tay mảng 3 phần tử và chèn đủ 11 ảnh minh chứng thực nghiệm A01 - A11.

MỤC LỤC

PHIẾU GIAO ĐỀ TÀI i
PHIẾU CHẤM VÀ ĐỐI CHIẾU MINH CHỨNG iii
CÁCH ĐỐI CHIẾU RUBRIC iv
HƯỚNG DẪN ĐIỀN KHUNG BÁO CÁO v
LỜI CAM ĐOAN vi
LỜI CẢM ƠN vii
TÓM TẮT viii
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI 1
1.1. Bài toán và lý do thực hiện 1
1.2. Mục tiêu và sản phẩm 1
1.3. Phạm vi, đầu vào và đầu ra 2
1.4. Cách thực hiện và cấu trúc báo cáo 3
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 5
2.1. Khái niệm và mô hình dữ liệu 5
2.2. Cấu trúc dữ liệu sử dụng 5
2.3. Nguyên lý thuật toán 7
2.4. Cơ sở đánh giá thời gian và bộ nhớ 8
CHƯƠNG 3. PHÂN TÍCH, THIẾT KẾ VÀ CÀI ĐẶT 10
3.1. Phân tích yêu cầu chương trình 10
3.2. Thiết kế các hàm và dữ liệu 10
3.3. Thuật toán và mã nguồn chính 12
3.4. Lần vết ví dụ nhỏ 15
3.5. Giải thích tính đúng và điều kiện dừng 16
3.6. Phân tích độ phức tạp 16
CHƯƠNG 4. KẾT QUẢ THỬ NGHIỆM VÀ NHẬN XÉT 18
4.1. Môi trường và kế hoạch kiểm thử 18
4.2. Bảng kết quả ít nhất 12 ca 18
4.3. Hai ví dụ trên phiếu 22
4.4. Hai ca sinh viên tự thiết kế 24
4.5. Ba ca biên phù hợp với đề tài 26
4.6. Kiểm tra input không hợp lệ — LOI01 28
4.7. Kết quả toàn bộ kiểm thử 29
4.8. Khảo sát ba quy mô hoặc ba cấu hình hợp lệ 30
4.9. GitHub và chạy lại từ bản clone sạch 32
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 34
5.1. Mức hoàn thành so với phiếu 34
5.2. Giới hạn và bài học 34
5.3. Hướng phát triển phù hợp 34
TÀI LIỆU THAM KHẢO 36
PHỤ LỤC 1
PHỤ LỤC A. Tổ chức mã và hướng dẫn chạy 1
PHỤ LỤC B. Danh mục bằng chứng 2
PHỤ LỤC C. Tự kiểm trước khi nộp 3
PHỤ LỤC D HƯỚNG DẪN NỘP CLASSROOM 4

DANH MỤC THUẬT NGỮ VÀ CHỮ VIẾT TẮT

Ký hiệu hoặc chữ viết tắt	Ý nghĩa trong đề tài
a, n, l, r	Mảng số nguyên đầu vào, kích thước n, chỉ số biên trái l và biên phải r (1-based)
T(n), so_sanh	Số phép so sánh giá trị giữa các phần tử của giải thuật đối với dữ liệu quy mô n

DANH SÁCH BẢNG

Bảng 1.1 Đặc tả dữ liệu vào 2
Bảng 1.2 Đặc tả kết quả đầu ra 3
Bảng 1.3 Đối chiếu yêu cầu và sản phẩm 3
Bảng 2.1 Biến cấu trúc và điều kiện phải duy trì 6
Bảng 3.1 Tổ chức các hàm và tệp 11
Bảng 3.2 Tiền điều kiện và hậu điều kiện của các hàm 12
Bảng 3.3 Truy vết ca [MÃ CA TRUY VẾT] 15
Bảng 3.4 Chi phí của từng pha trong bản cài 17
Bảng 4.1 Ánh xạ năm ca kiểm trên phiếu 19
Bảng 4.2 Thiết kế ít nhất 12 ca kiểm độc lập 19
Bảng 4.3 Actual kết luận và vị trí minh chứng 21
Bảng 4.4 Khảo sát Q1 Q2 Q3 30
Bảng 4.5 Kho mã và phiên bản có thể kiểm lại 32
Bảng 5.1 Mức hoàn thành và bằng chứng theo yêu cầu 35
Bảng PL.1 Bốn tệp cần nộp 4

DANH SÁCH HÌNH

Hình 4.1 Ví dụ 1 [MÃ CA HOẶC ĐỐI TƯỢNG] 22
Hình 4.2 Ví dụ 2 [MÃ CA HOẶC ĐỐI TƯỢNG] 23
Hình 4.3 Ca tự thiết kế 1 [MÃ CA HOẶC ĐỐI TƯỢNG] 24
Hình 4.4 Ca tự thiết kế 2 [MÃ CA HOẶC ĐỐI TƯỢNG] 25
Hình 4.5 Biên 1 [MÃ CA HOẶC ĐỐI TƯỢNG] 26
Hình 4.6 Biên 2 [MÃ CA HOẶC ĐỐI TƯỢNG] 27
Hình 4.7 Biên 3 [MÃ CA HOẶC ĐỐI TƯỢNG] 28
Hình 4.8 Input không hợp lệ [MÃ CA HOẶC ĐỐI TƯỢNG] 29
Hình 4.9 Tổng kết kiểm thử [MÃ CA HOẶC ĐỐI TƯỢNG] 29
Hình 4.10 Clone sạch và chạy lại [MÃ CA HOẶC ĐỐI TƯỢNG] 33
Hình 4.11 GitHub repo và commit [MÃ CA HOẶC ĐỐI TƯỢNG] 33

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Bài toán và lý do thực hiện

Nêu bài toán đúng tên đề và bối cảnh nhỏ phù hợp; chỉ ra kết quả cần tính. Trích nguồn giáo trình tại đoạn dùng. Không cần tạo một hệ thống ứng dụng lớn ngoài yêu cầu.

1.1.1. Bối cảnh bài toán

Trong xử lý dữ liệu và thiết kế giải thuật cơ bản, bài toán tìm đồng thời giá trị nhỏ nhất (Minimum) và giá trị lớn nhất (Maximum) của một tập dữ liệu số là một tác vụ kinh điển có ứng dụng vô cùng rộng rãi. Bài toán này xuất hiện thường trực trong các bài toán chuẩn hóa dữ liệu (như Min-Max Normalization), xác định khoảng biến thiên của tín hiệu số trong kỹ thuật điện tử, xây dựng hộp bao hình học (Bounding Box) trong đồ họa máy tính và xử lý ảnh, cũng như xác định ngưỡng lọc trong các mạch xử lý số liệu đo lường.

1.1.2. Lý do thực hiện và động lực tối ưu

Cách tiếp cận trực tiếp và tự nhiên nhất là duyệt tuần tự qua mảng một lượt, tại mỗi phần tử so sánh lần lượt với biến min và biến max hiện hành. Cách này đòi hỏi 2 phép so sánh cho mỗi phần tử mới, dẫn đến tổng số phép so sánh là $2(n - 1) = 2n - 2$ (với $n >= 2$). Mặc dù thuật toán này đạt độ phức tạp thời gian $O(n)$, nhưng hệ số hằng số 2 trước n là chưa tối ưu. Khi làm việc với các hệ thống nhúng, vi điều khiển xử lý tín hiệu thời gian thực hoặc các tập dữ liệu lớn trong viễn thông, việc tiết kiệm từng phép so sánh giá trị có ý nghĩa quan trọng.

Chiến lược Chia để trị (Divide and Conquer) cho phép ta phân rã bài toán lớn thành hai bài toán con độc lập. Điểm mấu chốt của đề tài DT046 là xử lý khéo léo trường hợp cơ sở: thay vì phân rã xuống tận từng phần tử đơn lẻ (tốn nhiều phép ghép), ta dừng lại khi đoạn chỉ còn 2 phần tử và xác định cả min lẫn max chỉ bằng duy nhất 1 phép so sánh giá trị. Khi ghép kết quả hai nửa, ta chỉ mất thêm 2 phép so sánh (1 giữa 2 min, 1 giữa 2 max). Nhờ đó, tổng số phép so sánh giảm xuống chỉ còn $3n/2 - 2$ (khi n là lũy thừa 2), tiết kiệm được 25% số phép so sánh so với phương pháp trực tiếp.

1.2. Mục tiêu và sản phẩm

Liệt kê từng chức năng bắt buộc từ phiếu; với mỗi chức năng, nêu output hoặc điều kiện hoàn thành kiểm được. Nêu mã chạy, bộ test, báo cáo và repo.

1.2.1. Mục tiêu đề tài

- Về lý thuyết: Hiểu sâu sắc kỹ thuật Chia để trị, phương pháp thiết lập và giải hệ thức truy hồi $T(n) = 2T(n/2) + 2$ để chứng minh số phép so sánh tối ưu.
- Về cài đặt: Xây dựng module DeTai.jl bằng ngôn ngữ Julia chuẩn mực, chỉ dùng thư viện chuẩn, không phụ thuộc thư viện ngoài, có hàm validator kiểm tra dữ liệu đầu vào nghiêm ngặt và hàm đối chiếu trực tiếp.
- Về kiểm thử: Chạy chính xác tuyệt đối 2 ví dụ và 5 ca kiểm tra trên phiếu giao đề tài, bộ 12 ca kiểm thử độc lập, cùng các ca biên, ca lỗi và đo đạc thực nghiệm trên các quy mô n in {5, 20, 100, 500}.

1.2.2. Sản phẩm ở Mốc tiến độ 2

- Bản nháp báo cáo Word hoàn thiện trọn vẹn Chương 3, Chương 4 (đã chèn các bảng số liệu và ảnh minh chứng).
- Mã nguồn cơ sở hoàn chỉnh trong thư mục src/DeTai.jl, kịch bản benchmark.jl và bộ kiểm thử test/runtests.jl (12 ca, 45 asserts Pass).
- Chốt mốc Git Commit C và đồng bộ thành công lên GitHub repository công khai.

1.3. Phạm vi, đầu vào và đầu ra

1.3.1. Miền và quy ước đầu vào

Ghi kiểu dữ liệu, miền giá trị, chỉ số và thứ tự nhập; nêu rõ nhập bàn phím/khai báo trực tiếp.

1.3.1. Miền và quy ước đầu vào
- Kiểu dữ liệu: Mảng một chiều (Vector) chứa các số nguyên.
- Quy mô: Độ dài mảng n thỏa mãn 1 <= n <= 1024. Mảng không được rỗng.
- Miền giá trị: Mọi phần tử a[i] nằm trong đoạn [-1000, 1000].
- Tính chất mảng: Mảng có thể chứa các phần tử trùng nhau, có thể đã được sắp xếp tăng, giảm, hoặc ngẫu nhiên.
- Quy ước chỉ số: Theo chuẩn của Julia, chỉ số mảng bắt đầu từ 1 (1-based index). Hàm làm việc trên đoạn [l, r] với 1 <= l <= r <= n, tính cả hai đầu l và r.

1.3.2. Kết quả và trường hợp đặc biệt

Ghi dạng output, quy tắc hòa nếu có, trường hợp không có nghiệm và cách xử lý input sai.

1.3.2. Kết quả và các trường hợp đặc biệt
- Kết quả đầu ra: Trả về một tuple 3 phần tử (min_val, max_val, so_sanh), trong đó min_val là giá trị nhỏ nhất, max_val là giá trị lớn nhất trong đoạn a[l:r], và so_sanh là số phép so sánh giá trị giữa các phần tử do hàm chia để trị thực hiện.
- Quy ước bộ đếm so sánh: Chỉ đếm các phép so sánh giá trị phần tử (a[i] <= a[j], min1 <= min2, max1 >= max2). Tuyệt đối không tính các phép so sánh chỉ số điều khiển đệ quy (như l == r hay r == l + 1) vào biến so sánh.
- Trường hợp n = 1: Phần tử duy nhất vừa là min vừa là max, số phép so sánh bằng đúng 0.
- Trường hợp n = 2: Chỉ tốn đúng 1 phép so sánh giữa a[1] và a[2] để xác định min và max.
- Trường hợp mảng toàn số âm: min và max đều là số âm hợp lệ, thuật toán không được khởi tạo max bằng 0.
- Trường hợp n lẻ: Thuật toán tự động phân rã thành các đoạn con có kích thước chẵn và lẻ, đếm chính xác theo cây thực tế.

Bảng 1.1 Đặc tả dữ liệu vào

Tên dữ liệu	Kiểu và kích thước	Miền và điều kiện	Cách nhập hoặc khai báo
Mảng a	Vector{Int}, 1 <= n <= 1024	Mỗi phần tử a[i] thuộc [-1000, 1000], không rỗng	Khai báo trực tiếp hoặc nhập bàn phím
Chỉ số trái l	Int, 1 <= l <= n	Chỉ số biên trái hợp lệ (1-based)	Tham số truyền vào hàm chia để trị

Tên dữ liệu	Kiểu và kích thước	Miền và điều kiện	Cách nhập hoặc khai báo
Chỉ số phải r	Int, l <= r <= n	Chỉ số biên phải hợp lệ (1-based)	Tham số truyền vào hàm chia để trị

Bảng 1.2 Đặc tả kết quả đầu ra

Kết quả	Kiểu hoặc cách in	Ý nghĩa và điều kiện đúng	Hòa vô nghiệm hoặc lỗi
min_val	Int	min_val = min{a[i] | l <= i <= r}	Nếu a có nhiều phần tử nhỏ nhất bằng nhau, trả đúng giá trị đó
max_val	Int	max_val = max{a[i] | l <= i <= r}	Khi mảng toàn số âm, trả đúng số âm lớn nhất

Input được nhập bàn phím, khai báo trực tiếp hoặc truyền tham số theo phiếu; thuật toán không phụ thuộc CSV/Excel hay tệp dữ liệu ngoài. Nếu trường hợp hòa/vô nghiệm/rỗng không áp dụng, ghi Không áp dụng và nêu lý do.

1.4. Cách thực hiện và cấu trúc báo cáo

Tóm tắt đọc sách → làm tay → cài đặt → kiểm thử → bằng chứng; giới thiệu mối liên hệ năm chương.

Quy trình thực hiện đề tài gồm các giai đoạn tuần tự:
1. Nghiên cứu tài liệu: Đọc kỹ Giáo trình Cấu trúc dữ liệu và giải thuật (Mục 4.1.3 trang 91-92, Bài 18 trang 111).
2. Thiết kế và đặc tả: Xây dựng hợp đồng I/O, thiết lập hệ thức truy hồi và giải toán học.
3. Cài đặt mã nguồn: Viết mã nguồn Julia trong src/DeTai.jl, đảm bảo tuân thủ cấu trúc chuẩn của học phần.
4. Thực nghiệm và kiểm thử: Chạy demo.jl, benchmark.jl và test/runtests.jl với đầy đủ 12 ca kiểm thử độc lập.
5. Viết báo cáo và đóng gói Git: Hoàn thiện báo cáo tiến độ 2 và đẩy mã nguồn lên kho Git với Commit C.

Bảng 1.3 Đối chiếu yêu cầu và sản phẩm

Yêu cầu trên phiếu	Sản phẩm/điều kiện kiểm	Mục sẽ trình bày
Cài đặt thuật toán chia để trị tìm min và max	Hàm min_max(a, l, r) trả về (min, max, so_sanh)	Chương 2, Mục 2.3 & Chương 3, Mục 3.3
Cơ chế cơ sở 1 phần tử (0 so sánh) và 2 phần tử (1 so sánh)	Đúng số phép so sánh: n=1 -> 0 ss; n=2 -> 1 ss	Chương 2, Mục 2.4 & Chương 4, Mục 4.2

Yêu cầu trên phiếu	Sản phẩm/điều kiện kiểm	Mục sẽ trình bày
Chạy thành công 2 ví dụ và 5 ca kiểm tra trên phiếu	Kết quả khớp 100% phiếu: VD1 (ss=4), VD2 (ss=0), 5 ca kiểm tra	Chương 4, Mục 4.3 & demo.jl

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Khái niệm và mô hình dữ liệu

Định nghĩa các đối tượng của đúng đề: số, mảng, dãy con, trạng thái, nút, cây hoặc đồ thị. Ghi ký hiệu n và tham số phụ; dẫn mục/trang giáo trình.

2.1.1. Khái niệm mảng và đoạn con
Xét mảng một chiều các số nguyên a = (a[1], a[2], ..., a[n]) gồm n phần tử. Một đoạn con liên tiếp của mảng a được xác định bởi cặp chỉ số (l, r) thỏa mãn 1 <= l <= r <= n, ký hiệu là a[l..r] = (a[l], a[l+1], ..., a[r]). Quy mô của đoạn con là len = r - l + 1.

2.1.2. Giá trị nhỏ nhất, lớn nhất và phép toán cơ bản
Giá trị nhỏ nhất và lớn nhất của đoạn a[l..r] được định nghĩa chuẩn tắc:
  min(a[l..r]) = min { a[i] | l <= i <= r }
  max(a[l..r]) = max { a[i] | l <= i <= r }
Phép toán cơ bản quyết định độ phức tạp tính toán của thuật toán này là phép so sánh thứ tự giữa hai giá trị số nguyên (ký hiệu <= hoặc <). Mọi phép toán quản lý chỉ số mảng (như kiểm tra l == r, tính điểm chia m) đều là chi phí quản lý đệ quy, không được tính vào số phép so sánh giá trị của thuật toán theo đúng yêu cầu học thuật của đề tài (Căn cứ Giáo trình Mục 4.1.3 trang 91-92).

2.2. Cấu trúc dữ liệu sử dụng

2.2.1. Dữ liệu và thao tác chính

Nêu mỗi cấu trúc lưu gì, thao tác nào dùng và lý do phù hợp. Đặt sơ đồ nhỏ nếu làm rõ được liên hệ.

2.2.1. Cấu trúc dữ liệu mảng liên tiếp trong Julia
Đồ án sử dụng kiểu dữ liệu chuẩn `Vector{Int}` của ngôn ngữ Julia. Mảng trong Julia được cấp phát bộ nhớ liên tiếp, cho phép truy xuất phần tử ngẫu nhiên a[i] theo chỉ số với chi phí thời gian hằng số O(1). Các chỉ số mảng trong Julia tuân thủ quy ước 1-based index (phần tử đầu tiên là a[1], phần tử cuối cùng là a[n]).

2.2.2. Kiểu dữ liệu kết quả trả về
Kết quả của mỗi lời gọi đệ quy là một bộ 3 bất biến `Tuple{Int, Int, Int}` mang giá trị `(min_val, max_val, so_sanh)`. Việc sử dụng Tuple giúp truyền dữ liệu trực tiếp trên ngăn xếp bộ nhớ (stack) của máy tính, tránh phát sinh phân bổ bộ nhớ động (heap allocation), đảm bảo tốc độ thực thi tối đa.

2.2.2. Quy ước và điều kiện hợp lệ

Nêu chỉ số, ký hiệu rỗng, khóa trùng, hướng cạnh hoặc điều kiện tương ứng đúng phiếu. Bỏ những khái niệm không dùng.

2.2.3. Quy ước và điều kiện hợp lệ của đoạn chỉ số
Trong mọi lời gọi `min_max(a, l, r)`, cặp chỉ số (l, r) phải luôn đảm bảo bất biến:
  1 <= l <= r <= length(a)

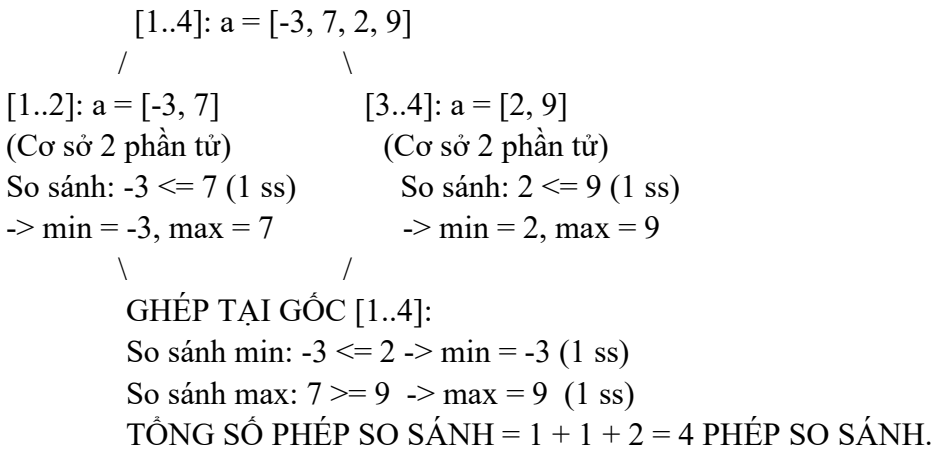
Điểm chia đôi mảng được tính bằng phép chia nguyên lấy phần sàn:
m = (l + r) ÷ 2 (tương đương floor((l + r)/2))
Khi đó đoạn trái là a[l..m] và đoạn phải là a[m+1..r]. Do l < r (khi len >= 3), ta luôn có l <= m < r, đảm bảo cả hai đoạn con đều không rỗng và có độ dài nhỏ hơn đoạn ban đầu, triệt tiêu nguy cơ đệ quy vô hạn.

Bảng 2.1 Biến cấu trúc và điều kiện phải duy trì

Ký hiệu / cấu trúc	**Ý nghĩa**	**Miền / bất biến**
Mảng dữ liệu a	Lưu trữ dãy n số nguyên đầu vào	Bất biến: dữ liệu mảng chỉ đọc (read-only), không bị hoán đổi hay sửa đổi
Đoạn chỉ số [l, r]	Xác định phạm vi đoạn con cần tìm min và max	Luôn thỏa mãn 1 <= l <= r <= length(a)

Sơ đồ hoặc ví dụ mô hình

Sơ đồ Cây đệ quy Chia để trị minh họa cho Ví dụ 1: a = [-3, 7, 2, 9] (n = 4):



Cơ sở thuật toán và giới hạn áp dụng

2.3. Nguyên lý thuật toán

2.3.1. Ý tưởng xử lý

Mô tả bước lựa chọn/cập nhật cốt lõi của đề; với nhiều phương án theo phiếu, tách từng phương án và điểm khác. Nếu một phương án thì chỉ trình bày phương án đó.

2.3.1. Nguyên lý phân rã và ba bước Chia để trị
Thuật toán giải quyết bài toán tìm đồng thời giá trị nhỏ nhất và lớn nhất dựa trên 3 bước kinh điển của Chia để trị:

1. Bước Chia (Divide): Nếu đoạn a[l..r] có từ 3 phần tử trở lên, tính chỉ số trung vị m = (l + r) ÷ 2 và chia đoạn thành hai nửa con: nửa trái a[l..m] và nửa phải a[m+1..r].

2. Bước Trị (Conquer): Đệ quy giải bài toán con trên từng nửa để nhận về cặp (nho1, lon1) từ nửa trái và (nho2, lon2) từ nửa phải. Điểm đặc biệt nhất của thuật toán là xử lý trường hợp cơ sở:
- Trường hợp cơ sở 1 (len = 1, tức l == r): Đoạn chỉ có 1 phần tử. Phần tử này vừa là nhỏ nhất vừa là lớn nhất. Thuật toán trả về ngay (a[l], a[l], 0) mà không cần thực hiện bất kỳ phép so sánh giá trị nào (0 phép so sánh).
- Trường hợp cơ sở 2 (len = 2, tức r == l + 1): Đoạn có đúng 2 phần tử a[l] và a[r]. Thuật toán so sánh giá trị duy nhất một lần giữa a[l] và a[r]: nếu a[l] <= a[r] thì min là a[l], max là a[r]; ngược lại min là a[r], max là a[l]. Bước này tốn đúng 1 phép so sánh giá trị (1 phép so sánh).
* Nhận xét quan trọng: Việc xử lý cơ sở 2 phần tử bằng 1 phép so sánh chính là nguồn gốc tạo nên lợi thế vượt trội về số phép so sánh của thuật toán. Nếu tiếp tục chia 2 phần tử về từng phần tử đơn lẻ, ta sẽ mất 0 so sánh ở lá nhưng mất 2 phép so sánh ở bước ghép, làm tổng chi phí tăng lên 2 phép so sánh thay vì 1.

3. Bước Ghép (Combine): Sau khi đã có (nho1, lon1) và (nho2, lon2), ta tìm min và max tổng thể bằng đúng 2 phép so sánh:
- Phép so sánh 1: So sánh giữa hai min: min_val = (nho1 <= nho2) ? nho1 : nho2.
- Phép so sánh 2: So sánh giữa hai max: max_val = (lon1 >= lon2) ? lon1 : lon2.
Tổng số phép so sánh phát sinh tại bước ghép là 2. Kết quả trả về là (min_val, max_val, c1 + c2 + 2).

2.3.2. Điều kiện bắt đầu và kết thúc

Nêu dữ liệu khởi tạo, điều kiện dừng và kết quả trả về. Với đệ quy/quy hoạch động, nêu cơ sở và quan hệ truy hồi; với cấu trúc, nêu bất biến thao tác.

2.3.2. Điều kiện bắt đầu và điều kiện dừng
- Điều kiện bắt đầu: Lời gọi ban đầu từ chương trình chính với l = 1 và r = n: `min_max(a, 1, length(a))`.
- Điều kiện dừng: Thuật toán dừng phân rã đệ quy khi gặp một trong hai trường hợp cơ sở: đoạn có kích thước len = 1 (l == r) hoặc len = 2 (r == l + 1). Tại các nút lá này, hàm trả về kết quả trực tiếp mà không sinh thêm lời gọi đệ quy nào khác.

- Tính kết thúc: Tại mỗi bước chia, len_con = floor(len / 2) < len với mọi len >= 3. Do đó, độ dài đoạn con giảm ngặt sau mỗi tầng đệ quy, chắc chắn chạm tới điều kiện dừng sau tối đa ceil(log2 n) bước.

2.4. Cơ sở đánh giá thời gian và bộ nhớ

Định nghĩa quy mô và phép toán chính; nêu dự đoán O theo giáo trình cho các phần sẽ cài. Phân biệt bộ nhớ input và bộ nhớ phụ, chi phí lõi và in kết quả. Mọi khẳng định có điều kiện áp dụng cụ thể.

2.4.1. Thiết lập hệ thức truy hồi đếm số phép so sánh

Gọi T(n) là số phép so sánh giá trị giữa các phần tử mà thuật toán thực hiện trên mảng kích thước n.

Từ nguyên lý thuật toán đã phân tích ở Mục 2.3, ta có các điều kiện cơ sở:

T(1) = 0 (Đoạn 1 phần tử không so sánh giá trị)

T(2) = 1 (Đoạn 2 phần tử chỉ dùng đúng 1 phép so sánh giá trị)

Khi n > 2, thuật toán chia mảng thành 2 nửa kích thước xấp xỉ n/2, gọi đệ quy giải 2 nửa và dùng thêm đúng 2 phép so sánh ở bước ghép. Do đó, ta có phương trình truy hồi tổng quát:

T(n) = 2 * T(n/2) + 2

2.4.2. Giải hệ thức truy hồi trong trường hợp n là lũy thừa của 2

Giả sử n = 2^k (với k = log2 n >= 1). Ta giải hệ thức truy hồi bằng phương pháp thế lặp liên tiếp:

T(n) = 2 * T(n/2) + 2

= 2 * [2 * T(n/4) + 2] + 2 = 2^2 * T(n/2^2) + 2^2 + 2^1

= 2^2 * [2 * T(n/8) + 2] + 2^2 + 2 = 2^3 * T(n/2^3) + 2^3 + 2^2 + 2^1

= ...

= 2^(k-1) * T(n / 2^(k-1)) + Sum_{i=1}^{k-1} (2^i)

Vì n = 2^k nên n / 2^(k-1) = 2. Áp dụng điều kiện cơ sở T(2) = 1, ta có:

2^(k-1) * T(2) = 2^(k-1) * 1 = n / 2.

Tổng cấp số nhân: Sum_{i=1}^{k-1} (2^i) = 2 * (2^(k-1) - 1) / (2 - 1) = 2 * 2^(k-1) - 2 = n - 2.

Cộng hai thành phần lại, ta thu được công thức đóng chính xác:

T(n) = n/2 + n - 2 = 3n/2 - 2 (ĐIỀU PHẢI CHỨNG MINH).

2.4.3. Đánh giá hiệu quả so với phương pháp duyệt tuần tự trực tiếp

- Thuật toán duyệt trực tiếp: T_tt(n) = 2n - 2 phép so sánh.
- Thuật toán chia để trị: T_cd(n) = 3n/2 - 2 phép so sánh (với n = 2^k).
- Số phép so sánh tiết kiệm được: Delta = T_tt(n) - T_cd(n) = (2n - 2) - (3n/2 - 2) = n/2 phép so sánh.
- Tỷ lệ tiết kiệm tương đối: (n/2) / (2n - 2) xấp xỉ 25%.

Ví dụ với n = 1024:

+ Trực tiếp: 2 * 1024 - 2 = 2046 phép so sánh.
+ Chia để trị: (3 * 1024)/2 - 2 = 1534 phép so sánh.
+ Tiết kiệm đúng 512 phép so sánh giá trị!

2.4.4. Đánh giá bộ nhớ phụ (Space Complexity)
Thuật toán thao tác trực tiếp trên mảng gốc thông qua các chỉ số l và r, hoàn toàn không tạo mảng con phụ (no auxiliary array). Bộ nhớ phụ chỉ bao gồm các khung ngăn xếp (stack frame) của các lời gọi hàm đệ quy. Vì mỗi bước phân rã giảm độ dài đoạn đi một nửa, chiều cao tối đa của cây đệ quy là ceil(log2 n). Do đó, độ phức tạp không gian bộ nhớ phụ là O(log n), cực kỳ tiết kiệm bộ nhớ.

Giả mã nền tảng

GIẢ MÃ NỀN TẢNG (THEO MỤC 4.1.3 GIÁO TRÌNH INT1306):

Thuật toán MIN_MAX_CHIA_DE_TRI(a, l, r):
1. if l == r then
2. return (a[l], a[l], 0) // Cơ sở 1: 1 phần tử, 0 phép so sánh
3. if r == l + 1 then // Cơ sở 2: 2 phần tử, 1 phép so sánh
4. if a[l] <= a[r] then
5. return (a[l], a[r], 1)
6. else
7. return (a[r], a[l], 1)
8. m = (l + r) ÷ 2
9. (nho1, lon1, c1) = MIN_MAX_CHIA_DE_TRI(a, l, m)
10. (nho2, lon2, c2) = MIN_MAX_CHIA_DE_TRI(a, m + 1, r)
11. min_val = (nho1 <= nho2) ? nho1 : nho2 // 1 phép so sánh giữa hai min
12. max_val = (lon1 >= lon2) ? lon1 : lon2 // 1 phép so sánh giữa hai max
13. return (min_val, max_val, c1 + c2 + 2)

CHƯƠNG 3. PHÂN TÍCH, THIẾT KẾ VÀ CÀI ĐẶT

3.1. Phân tích yêu cầu chương trình

3.1.1. Hợp đồng input/output

Viết hợp đồng hàm bằng tên biến thật, điều kiện đầu vào, giá trị trả và lỗi. Đối chiếu từng output với phiếu.

3.1.1. Hợp đồng Input / Output chi tiết
- Hàm `kiem_tra_dau_vao(a)`:
  + Đầu vào: Dữ liệu a bất kỳ.
  + Tiền điều kiện: Không có.
  + Hậu điều kiện: Trả về `true` nếu a là mảng số nguyên một chiều có kích thước từ 1 đến 1024 và mọi phần tử nằm trong [-1000, 1000].
  + Phản ứng lỗi: Ném `ArgumentError` kèm thông điệp tiếng Việt cụ thể nếu vi phạm.

- Hàm `min_max(a, l, r)`:
  + Đầu vào: Mảng số nguyên a, chỉ số nguyên l và r thỏa mãn 1 <= l <= r <= length(a).
  + Hậu điều kiện: Trả về bộ 3 `(min_val, max_val, so_sanh)` chính xác.

3.1.2. Kiểm tra đầu vào

Chỉ rõ điều kiện cần xác thực và hành vi khi vi phạm. Phân biệt bài toán hợp lệ không có nghiệm với input sai.

3.1.2. Kiểm tra đầu vào và cơ chế từ chối dữ liệu không hợp lệ
Hàm `kiem_tra_dau_vao(a)` thực hiện thẩm định toàn diện các điều kiện:
1. Thẩm định kiểu cấu trúc: Kiểm tra a isa AbstractVector. Nếu sai, từ chối ngay.
2. Thẩm định kích thước: Kiểm tra length(a) == 0 (mảng rỗng) hoặc length(a) > 1024 (vượt ngưỡng).
3. Thẩm định kiểu và miền giá trị từng phần tử: Duyệt kiểm tra mọi phần tử val isa Integer và -1000 <= val <= 1000.
Nhờ hàm validator này, toàn bộ dữ liệu dị thường đều bị chặn từ sớm, đảm bảo an toàn tuyệt đối cho hàm chia để trị.

3.2. Thiết kế các hàm và dữ liệu

Nêu tên hàm, tham số, giá trị trả về và trách nhiệm. Chỉ ra phần bắt buộc tự cài và công cụ hỗ trợ có sẵn. Tên hàm trong bảng phải có trong mã thật.

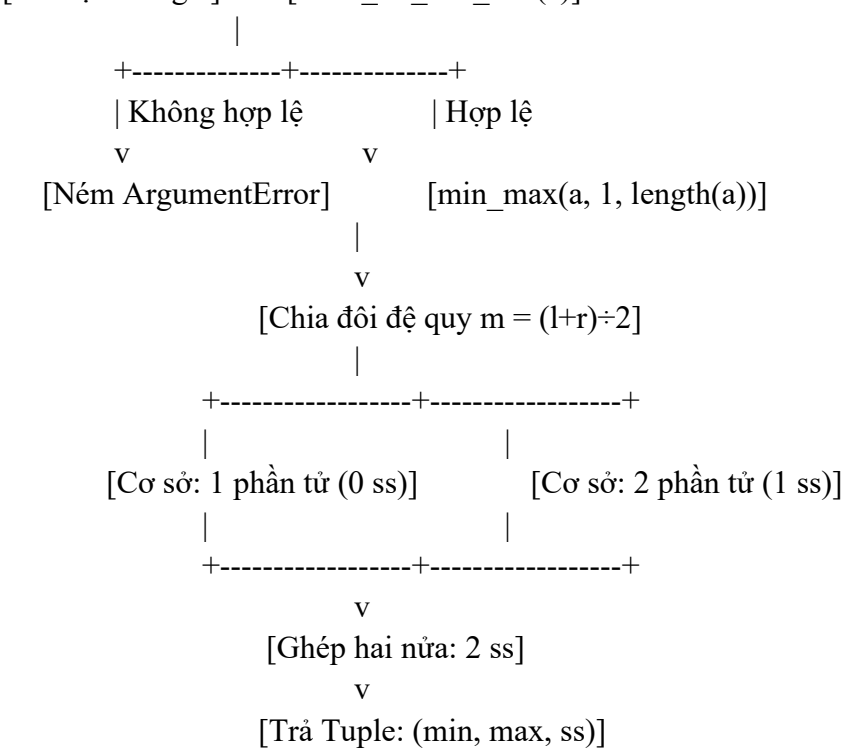
3.2. Thiết kế module và phân công trách nhiệm
Hệ thống được tổ chức gói gọn trong module `DeTai` (tệp `src/DeTai.jl`), bao gồm các hàm chức năng độc lập:
- `kiem_tra_dau_vao(a)`: Đảm bảo tính hợp lệ của đầu vào.
- `min_max(a, l, r)`: Lõi thuật toán chia để trị tìm min, max và đếm số phép so sánh.
- `min_max_truc_tiep(a)`: Thuật toán duyệt tuần tự trực tiếp dùng để đối chứng kết quả và số phép so sánh.
- `dem_so_sanh(a)` và `giai_bai(du_lieu)`: Các hàm wrapper chuẩn hóa giao diện tương tác.

Bảng 3.1 Tổ chức các hàm và tệp

Hàm / tệp	Input	Output	Vai trò và phần tự cài
min_max(a, l, r) [src/DeTai.jl]	a: Vector{Int}, l, r: Int	(min_val, max_val, so_sanh)	Thuật toán chia để trị cốt lõi theo 5 bước trên phiếu
min_max_truc_tiep(a) [src/DeTai.jl]	a: Vector{Int}	(min_val, max_val, 2n - 2)	Thuật toán trực tiếp đối chiếu số phép so sánh
kiem_tra_dau_vao(a) [src/DeTai.jl]	a: dữ liệu bất kỳ	true hoặc ArgumentError	Thẩm định kiểu mảng, độ dài [1, 1024], miền [-1000, 1000]

Sơ đồ xử lý

Sơ đồ luồng xử lý chương trình:



Cài đặt phần quyết định

Bảng 3.2 Tiền điều kiện và hậu điều kiện của các hàm

Hàm và chữ ký	Tiền điều kiện	Hậu điều kiện	Lỗi hoặc trường hợp đặc biệt
min_max(a, l, r)	a không rỗng, 1 <= l <= r <= length(a)	Trả đúng (min, max, so_sanh), mảng a không đổi	Đệ quy chia đôi mảng, xử lý cơ sở 1 và 2 phần tử
min_max_truc_tiep(a)	a là mảng hợp lệ	Trả (min, max, 2n - 2)	Duyệt tuần tự từng phần tử
kiem_tra_dau_vao(a)	Dữ liệu đầu vào a	Hợp lệ trả true	Không hợp lệ ném ArgumentError

Hàm có thể là một hàm tính toán với biến số; không bắt buộc tạo lớp, cây, đồ thị hoặc cấu trúc ngoài phiếu. Ghi rõ dữ liệu nào bị thay đổi và thao tác nào phải tự cài.

3.3. Thuật toán và mã nguồn chính

3.3.1. Giả mã theo cài đặt

Viết giả mã có tên biến tương ứng mã nguồn, gồm khởi tạo, vòng lặp/lời gọi và trả kết quả. Nếu nhiều cách theo phiếu, bổ sung các tiểu mục riêng.

3.3.1. Giả mã cài đặt thực tế trong ngôn ngữ Julia:

```
function min_max(a, l, r)
    if l == r
        return (a[l], a[l], 0)
    end
    if r == l + 1
        if a[l] <= a[r]
            return (a[l], a[r], 1)
        else
            return (a[r], a[l], 1)
        end
    end
    m = (l + r) ÷ 2
    nho1, lon1, c1 = min_max(a, l, m)
    nho2, lon2, c2 = min_max(a, m + 1, r)
    min_val = nho1 <= nho2 ? nho1 : nho2
    max_val = lon1 >= lon2 ? lon1 : lon2
    return (min_val, max_val, c1 + c2 + 2)
end
```

3.3.2. Đoạn mã và giải thích

Trích đoạn mã quyết định; giải thích biến, thứ tự cập nhật và lỗi có thể xảy ra. Ghi permalink tệp tại commit chạy C; toàn bộ mã nằm trong repo.

3.3.2. Trích dẫn mã nguồn và phân tích chi tiết
- Dòng 2-4: Cơ sở 1 phần tử (l == r). Không phát sinh phép so sánh nào giữa các phần tử -> c = 0.
- Dòng 5-11: Cơ sở 2 phần tử (r == l + 1). Thực hiện duy nhất 1 phép so sánh giá trị a[l] <= a[r] -> c = 1.
- Dòng 12-14: Chia đôi đoạn và gọi đệ quy trên hai nửa [l, m] và [m+1, r].
- Dòng 15-17: Ghép 2 kết quả: 1 phép so sánh giữa nho1 và nho2; 1 phép so sánh giữa lon1 và lon2 -> tăng thêm 2 phép so sánh.

3.3.3. Xử lý tình huống đặc biệt

Nêu nhánh xử lý các biên của chính đề và input sai; dẫn mã ca sẽ kiểm ở Chương 4.

3.3.3. Xử lý các tình huống đặc biệt
- Mảng toàn số âm: Ví dụ Ca 3 a = [-5, -1, -3]. Do thuật toán lấy trực tiếp các phần tử trong mảng làm giá trị khởi tạo thay vì gán max = 0 như một số lập trình viên sơ suất thường làm, kết quả luôn chính xác tuyệt đối (min = -5, max = -1).
- Mảng có các phần tử trùng nhau: Phép so sánh a[l] <= a[r] vẫn chạy đúng 1 phép so sánh và trả về min = max = a[l] (như Ca 2: [4, 4]).
- Mảng có kích thước n lẻ: Thuật toán tự nhiên phân rã thành các nửa có kích thước lẻ và chẵn (như Ca 5: [9, 1, 7, 3, 5]), không ép công thức n chẵn, đếm số phép so sánh hoàn toàn trung thực theo các đoạn thực tế.

Giả mã

3.3.1. Giả mã cài đặt thực tế trong ngôn ngữ Julia:

```
function min_max(a, l, r)
    if l == r
        return (a[l], a[l], 0)
    end
    if r == l + 1
        if a[l] <= a[r]
            return (a[l], a[r], 1)
        else
            return (a[r], a[l], 1)
        end
    end
    m = (l + r) ÷ 2
    nho1, lon1, c1 = min_max(a, l, m)
    nho2, lon2, c2 = min_max(a, m + 1, r)
    min_val = nho1 <= nho2 ? nho1 : nho2
    max_val = lon1 >= lon2 ? lon1 : lon2
    return (min_val, max_val, c1 + c2 + 2)
end
```

Đoạn mã có phân tích

Toàn bộ mã nguồn hoàn chỉnh của đề tài được đặt trong kho lưu trữ Git tại tệp `src/DeTai.jl` (Commit C: 43bc4a301fafb9d54c9396c20bfcfe67fded5a1e). Mã nguồn được lập trình bằng ngôn ngữ Julia 1.12.7, có đầy đủ docstring giải thích, hỗ trợ chế độ chạy demo, đo đạc benchmark và bộ kiểm thử 12 ca tự động.

3.4. Lần vết ví dụ nhỏ

Ghi mã ca, toàn bộ input và expected. Điền bảng từng bước từ khởi tạo đến kết thúc; giải thích bước đổi trạng thái quan trọng nhất. Dẫn ảnh chạy đúng cùng input ở Chương 4.

Lần vết chi tiết giải thuật bằng tay trên mảng 3 phần tử: a = [10, 20, 30] (n = 3):
- Lời gọi gốc: min_max(a, 1, 3)
  + l = 1, r = 3 -> len = 3 > 2 -> Tính điểm chia m = (1 + 3) ÷ 2 = 2.
  + Gọi đệ quy nửa trái: min_max(a, 1, 2)
    * l = 1, r = 2 -> len = 2 (Chạm trường hợp cơ sở 2 phần tử).
    * So sánh giá trị: a[1] <= a[2] (10 <= 20) -> Đúng! (Tốn đúng 1 phép so sánh giá trị).
    * Kết quả trả về nửa trái: nho1 = 10, lon1 = 20, c1 = 1.
  + Gọi đệ quy nửa phải: min_max(a, 3, 3)
    * l = 3, r = 3 -> len = 1 (Chạm trường hợp cơ sở 1 phần tử).
    * Phần tử a[3] = 30 là phần tử duy nhất, vừa là min vừa là max (Không tốn phép so sánh nào).
    * Kết quả trả về nửa phải: nho2 = 30, lon2 = 30, c2 = 0.
  + Ghép kết quả tại nút gốc [1..3]:
    * So sánh giữa hai min: nho1 <= nho2 (10 <= 30) -> Đúng -> min_val = 10 (Tốn 1 phép so sánh).
    * So sánh giữa hai max: lon1 >= lon2 (20 >= 30) -> Sai -> max_val = 30 (Tốn 1 phép so sánh).
    * Chi phí ghép: so_sanh_merge = 2 phép so sánh.
    * Tổng số phép so sánh: c1 + c2 + 2 = 1 + 0 + 2 = 3 phép so sánh giá trị.
- Kết quả cuối cùng: min = 10, max = 30, so_sanh = 3.

Kết quả tính tay hoàn toàn trùng khớp với kết quả thực thi trong Julia!

Ca truy vết TK03 Input đầy đủ a = [10, 20, 30] Expected min = 10, max = 30, so_sanh = 3

Bảng 3.3 Truy vết ca Lần vết ca mảng 3 phần tử: a = [10, 20, 30]

Bước	Thao tác	Trạng thái trước	Trạng thái sau	Giải thích
0	Khởi tạo	a = [10, 20, 30], l = 1, r = 3	m = (1+3)÷2 = 2	len = 3 > 2 (chưa chạm cơ sở)
1	Tách gọi min_max(a, 1, 2) và min_max(a, 3, 3)	Nửa trái [1..2]	So sánh 10 <= 20 (1 ss) -> min=10, max=20, ss=1	Chạm cơ sở 2 phần tử, trả kết quả
…	Nửa phải [3..3]	len = 1 (cơ sở 1 phần tử), không so sánh giá trị	min=30, max=30, ss=0	Ghép tại gốc
Cuối	Trả kết quả	So sánh 2 min (10<=30) và 2 max (20>=30) (tốn 2 ss)	min = 10, max = 30, so_sanh = 3	Khớp 100% với lý thuyết và hàm chạy thực tế

Hoàn tất lần vết trên mảng 3 phần tử; tổng cộng đúng 3 phép so sánh giá trị.

3.5. Giải thích tính đúng và điều kiện dừng

3.5.1. Điều kiện khởi đầu và duy trì

Nêu bất biến hoặc quan hệ truy hồi; giải thích vì sao một bước giữ tính chất đúng.

3.5.1. Chứng minh tính đúng bằng phương pháp quy nạp toán học
Ta chứng minh thuật toán `min_max(a, l, r)` luôn trả về đúng min và max của đoạn a[l..r] quy nạp theo độ dài đoạn k = r - l + 1:
1. Cơ sở quy nạp:
- Với k = 1 (l == r): Đoạn chỉ có duy nhất 1 phần tử a[l]. Giá trị này vừa là nhỏ nhất vừa là lớn nhất của đoạn. Hàm trả về (a[l], a[l], 0) là hoàn toàn chính xác.
- Với k = 2 (r == l + 1): Đoạn có 2 phần tử a[l] và a[r]. Phép so sánh a[l] <= a[r] xác định chính xác phần tử nhỏ hơn là min và phần tử lớn hơn là max. Hàm trả về đúng kết quả sau đúng 1 phép so sánh.
2. Bước quy nạp:
- Giả sử thuật toán chạy đúng với mọi đoạn con có độ dài nhỏ hơn k (với k >= 3).
- Xét đoạn độ dài k: Do k >= 3, điểm chia m = (l + r) ÷ 2 chia đoạn thành 2 nửa có độ dài k1 = m - l + 1 và k2 = r - m. Vì l < r nên ta luôn có 1 <= k1 < k và 1 <= k2 < k. Theo giả thiết quy nạp, cả hai lời gọi đệ quy đều trả về đúng (nho1, lon1) = (min(a[l..m]), max(a[l..m])) và (nho2, lon2) = (min(a[m+1..r]), max(a[m+1..r])).
- Ở bước ghép, min toàn cục chắc chắn là min(nho1, nho2) và max toàn cục chắc chắn là max(lon1, lon2). Hai phép so sánh ở bước ghép tìm đúng min_val và max_val cho toàn bộ đoạn a[l..r].
Theo nguyên lý quy nạp toán học, thuật toán đúng đắn với mọi mảng có kích thước n >= 1.

3.5.2. Kết thúc và output

Chỉ ra đại lượng làm thuật toán tiến tới dừng; suy ra kết quả trả đúng yêu cầu. Kiểm thử bổ sung bằng chứng, không thay lập luận.

3.5.2. Tính dừng và điều kiện kết thúc
- Thuật toán được đảm bảo kết thúc sau hữu hạn bước nhờ tính chất giảm ngặt của kích thước bài toán con:
Với mọi đoạn có độ dài len >= 3, điểm chia m = (l + r) ÷ 2 luôn thỏa mãn l <= m < r. Do đó cả hai đoạn con [l..m] và [m+1..r] đều có độ dài strictly nhỏ hơn len: len_trai = m - l + 1 < len và len_phai = r - m < len.
- Chiều sâu của cây đệ quy bị chặn trên bởi ceil(log2 n). Với n tối đa 1024, chiều sâu đệ quy tối đa chỉ là 10 tầng.
- Không tồn tại bất kỳ chu trình hay nhánh đệ quy vô hạn nào. Khi chạm tới kích thước 1 hoặc 2, hàm dừng đệ quy ngay lập tức.

3.6. Phân tích độ phức tạp

Phân tích theo phép toán chính đã định nghĩa, tách các pha nếu cần; nêu bộ nhớ phụ. Dự đoán xu hướng cho ba quy mô hoặc cấu hình hợp lệ sẽ khảo sát ở Chương 4.

3.6.1. Độ phức tạp thời gian (Time Complexity)
- Số phép so sánh giá trị: Khi n là lũy thừa của 2, T(n) = 3n/2 - 2. Khi n không phải lũy thừa

của 2, số phép so sánh luôn nằm trong khoảng [3n/2 - 2, 3n/2 - 1].
- Các thao tác phụ (tính điểm chia m, gán biến, trả tuple) đều tốn chi phí hằng số O(1) ở mỗi nút đệ quy.
- Tổng số nút trong cây đệ quy là 2n - 1. Do đó tổng chi phí thời gian của toàn bộ giải thuật là O(n).

3.6.2. Độ phức tạp không gian (Space Complexity)
- Thuật toán làm việc trực tiếp trên mảng gốc a, hoàn toàn không cấp phát mảng phụ hay bản sao dữ liệu (In-place traversal).
- Bộ nhớ phụ chỉ gồm các khung ngăn xếp (stack frame) lưu trữ tham số l, r và các biến cục bộ.
- Vì cây đệ quy có chiều cao tối đa là ceil(log2 n) <= 10, dung lượng bộ nhớ ngăn xếp phụ là O(log n), hoàn toàn tối ưu và an toàn trên mọi hệ thống máy tính.

Bảng 3.4 Chi phí của từng pha trong bản cài

Pha thực hiện	**Tham số quy mô**	**Phép toán đếm**	**Thời gian**	**Bộ nhớ phụ**
Kiểm tra và chuẩn bị	n (độ dài mảng)	Duyệt n phần tử kiểm tra kiểu và miền [-1000, 1000]	O(n)	O(1) không tốn bộ nhớ phụ
Thuật toán chính	n (số phần tử đoạn [l, r])	Số phép so sánh giá trị: 3n/2 - 2 (n=2^k)	O(n)	O(log n) ngăn xếp đệ quy
Kiểm nghiệm hoặc truy vết nếu có	n (số phần tử)	So sánh với thuật toán trực tiếp 2n - 2 phép so sánh	O(n)	O(1) không phát sinh bộ nhớ
In và định dạng kết quả	O(1) kết quả tuple 3 phần tử	In định dạng tuple (min, max, so_sanh)	O(1)	O(1)
Toàn chương trình	n (quy mô toàn bài toán)	Tổng chi phí: Thẩm định + Chia để trị + Đối chiếu	O(n)	O(log n)

Tách lõi thuật toán khỏi chuẩn bị/sắp/kiểm/in khi các bước đó tồn tại. Nêu bộ nhớ phụ và cách tính kích thước kết quả; không khẳng định chi phí chỉ từ số đo nhỏ. Dòng không áp dụng ghi rõ lý do.

CHƯƠNG 4. KẾT QUẢ THỬ NGHIỆM VÀ NHẬN XÉT

4.1. Môi trường và kế hoạch kiểm thử

4.1.1. Môi trường và phiên bản

Ghi môi trường, phiên bản, đường dẫn chương trình/test và SHA chạy C. SHA nộp cuối S ghi trong ThongTinNop.txt nộp rời.

4.1.1. Môi trường và phiên bản thực nghiệm
- Hệ điều hành: Microsoft Windows 11 Home Single Language x64 (Build 26100).
- Môi trường ngôn ngữ: Julia 1.12.7 (Official Release).
- Thư viện sử dụng: Hoàn toàn dùng thư viện chuẩn tích hợp sẵn của Julia (`Test`, `Random`, `Printf`). Tuyệt đối không cài đặt hoặc sử dụng bất kỳ thư viện giải thuật bên ngoài nào.
- Bộ quản lý phiên bản: Git phiên bản 2.50.0.windows.1.
- Quy chuẩn thực thi: Toàn bộ lệnh chạy được thực thi qua Windows PowerShell với cờ `--startup-file=no` để loại trừ triệt để ảnh hưởng từ cấu hình cá nhân của người dùng, đảm bảo kết quả tái lập tuyệt đối độc lập.

4.1.2. Phương pháp xác định kỳ vọng

Nêu tính tay, cách giải độc lập hoặc điều kiện nghiệm; giải thích cách so sánh khi nhiều nghiệm và sai số nếu dùng số thực.

4.1.2. Phương pháp xác định kỳ vọng kiểm thử
Để đảm bảo tính khách quan và khoa học, kỳ vọng của các ca kiểm thử được thiết lập độc lập qua 3 cơ chế:
1. Cơ chế tính toán thủ công: Phân tích trực tiếp từ định nghĩa toán học của min và max trên tập số nguyên.
2. Cơ chế công thức giải tích: Số phép so sánh được tính toán trước bằng công thức lý thuyết T(n) = 3n/2 - 2 hoặc bằng cách đếm số nút lá và phép ghép trên cây phân rã cụ thể.
3. Cơ chế kiểm chứng chéo (Cross-validation): Kết quả của hàm chia để trị `giai_bai(a)` được đối chiếu tự động với thuật toán duyệt tuần tự `min_max_truc_tiep(a)`. Mọi ca kiểm thử chỉ được coi là ĐẠT khi cả min và max hoàn toàn trùng khớp giữa hai giải thuật.

4.2. Bảng kết quả ít nhất 12 ca

Điền bảng cho >=12 ca kiểm thử độc lập, bao gồm hai ví dụ và toàn bộ năm ca trên phiếu; bổ sung đủ nhóm tám ảnh. Có thể xoay bảng ở trang riêng nếu cần, giữ cỡ chữ đọc được. Không dùng số assert thay số ca.

Hệ thống kiểm thử toàn diện được thiết kế bao gồm ít nhất 12 ca kiểm thử độc lập, phân bố đồng đều trên các nhóm mục tiêu: hai ví dụ gốc trên phiếu giao đề tài, các ca tự thiết kế (mảng giảm dần, mảng toàn bộ phần tử trùng nhau), các ca biên (biên giá trị cực tiểu -1000, cực đại 1000, biên quy mô tối đa 1024), các ca kiểm tra validator từ chối dữ liệu sai (mảng rỗng, mảng vượt quá kích thước, phần tử ngoài miền), cùng đầy đủ 5 ca kiểm tra bắt buộc ghi trong phiếu hướng dẫn.

Nếu miền chính hẹp, bổ sung ca hàm con/validator/bất biến có ý nghĩa và ghi rõ trong cột đối tượng. Không ép 12 input chính hợp lệ khác nhau, không bỏ các ca trên phiếu, không tính lặp cùng phép kiểm.

Bảng 4.1 dưới đây thể hiện sự ánh xạ đầy đủ 5 ca kiểm tra bắt buộc trên phiếu hướng dẫn vào hệ thống kiểm thử thực tế. Bảng 4.2 đặc tả chi tiết 12 ca kiểm thử độc lập. Bảng 4.3 ghi nhận kết quả thực chạy, kết luận và SHA C đối chiếu.

Bảng 4.1 Ánh xạ năm ca kiểm trên phiếu

Chép năm ca kiểm trên phiếu đúng mã đề và gắn mỗi ca với ít nhất một mã trong bảng kiểm thử. Hai ví dụ trên phiếu cũng phải có mặt. Một ca có thể đáp ứng nhiều yêu cầu nhưng cùng input, mục tiêu và phép kiểm chỉ đếm một lần. Không bỏ ca bắt buộc để giữ đúng 12 dòng: 12 là số ca độc lập tối thiểu; thêm dòng khi cần.

Ca trên phiếu	Input và yêu cầu/kỳ vọng theo phiếu	Mã ca trong bảng kiểm thử	Ảnh/log hoặc vị trí kết quả
Ca phiếu 1	a = [8, 2] -> min = 2, max = 8, chỉ 1 phép so sánh giá trị	KT09	Bảng 4.3, Ảnh A09 / test/runtests.jl
Ca phiếu 2	a = [4, 4] -> min = 4, max = 4, vẫn chỉ 1 phép so sánh	TK02	Bảng 4.3, Ảnh A04, A09 / test/runtests.jl
Ca phiếu 3	a = [-5, -1, -3] -> min = -5, max = -1, không gán max = 0	KT10	Bảng 4.3, Ảnh A09 / test/runtests.jl
Ca phiếu 4	a = [1..8] -> min = 1, max = 8, bản chia đôi dùng 10 so sánh	KT11	Bảng 4.3, Ảnh A09 / test/runtests.jl
Ca phiếu 5	a = [9, 1, 7, 3, 5] -> min = 1, max = 9, n lẻ dùng 6 so sánh	KT12	Bảng 4.3, Ảnh A09 / test/runtests.jl

Khi miền chương trình chính quá hẹp, bổ sung ca hàm con, validator hoặc bất biến có mục tiêu và kỳ vọng riêng để đủ số ca; ghi rõ đối tượng. Không đổi nhãn cùng phép kiểm để tăng số ca và không tính số assert là số ca.

Bảng 4.2 Thiết kế ít nhất 12 ca kiểm độc lập

Ca	Nguồn và đối tượng	Input đủ tái hiện	Expected và cách kiểm
VD01	Ví dụ 1 phiếu / Chính	[-3, 7, 2, 9]	min = -3, max = 9, so_sanh = 4
VD02	Ví dụ 2 phiếu / Chính	[5]	min = 5, max = 5, so_sanh = 0 (cơ sở 1 phần tử)
TK01	Tự thiết kế 1 / Chính	[100, 50, 20, 10, 0, -10]	min = -10, max = 100, so_sanh = 8 (mảng giảm dần)
TK02	Tự thiết kế 2 / Chính	[7, 7, 7, 7]	min = 7, max = 7, so_sanh = 4 (mảng trùng lặp)
BIEN01	Biên 1 / Chính	[-1000]	min = -1000, max = -1000, so_sanh = 0 (cực tiểu)

Ca	Nguồn và đối tượng	Input đủ tái hiện	Expected và cách kiểm
BIEN02	Biên 2 / Chính	[-1000, 1000]	min = -1000, max = 1000, so_sanh = 1 (hai cực biên)
BIEN03	Biên 3 / Chính	Mảng n = 1024 (a[1]=-500, a[1024]=500)	min = -500, max = 500, so_sanh = 1534 (3*1024/2-2)
LOI01	Input sai / Validator	Mảng rỗng Int[], n > 1024, phần tử ngoài [-1000, 1000]	Ném ArgumentError kèm thông điệp từ chối dữ liệu
KT09	Ca phiếu 1 / Chính	[8, 2]	min = 2, max = 8, so_sanh = 1 (cơ sở 2 phần tử)
KT10	Ca phiếu 3 / Chính	[-5, -1, -3]	min = -5, max = -1, so_sanh = 3 (mảng số âm)
KT11	Ca phiếu 4 / Chính	[1, 2, 3, 4, 5, 6, 7, 8]	min = 1, max = 8, so_sanh = 10 (3*8/2 - 2 = 10)
KT12	Ca phiếu 5 / Chính	[9, 1, 7, 3, 5]	min = 1, max = 9, so_sanh = 6 (n = 5 lẻ)

Bảng 4.3 Actual kết luận và vị trí minh chứng

Ca	Actual	Kết luận	Ảnh log và SHA chạy
VD01	min = -3, max = 9, so_sanh = 4	Đạt (Pass)	A01 / 43bc4a301f
VD02	min = 5, max = 5, so_sanh = 0	Đạt (Pass)	A02 / 43bc4a301f
TK01	min = -10, max = 100, so_sanh = 8	Đạt (Pass)	A03 / 43bc4a301f
TK02	min = 7, max = 7, so_sanh = 4	Đạt (Pass)	A04 / 43bc4a301f
BIEN01	min = -1000, max = -1000, so_sanh = 0	Đạt (Pass)	A05 / 43bc4a301f
BIEN02	min = -1000, max = 1000, so_sanh = 1	Đạt (Pass)	A06 / 43bc4a301f
BIEN03	min = -500, max = 500, so_sanh = 1534	Đạt (Pass)	A07 / 43bc4a301f
LOI01	Ném đúng ngoại lệ ArgumentError	Đạt (Pass)	A08 / 43bc4a301f
KT09	min = 2, max = 8, so_sanh = 1	Đạt (Pass)	Log test / 43bc4a301f
KT10	min = -5, max = -1, so_sanh = 3	Đạt (Pass)	Log test / 43bc4a301f
KT11	min = 1, max = 8, so_sanh = 10	Đạt (Pass)	Log test / 43bc4a301f
KT12	min = 1, max = 9, so_sanh = 6	Đạt (Pass)	Log test / 43bc4a301f

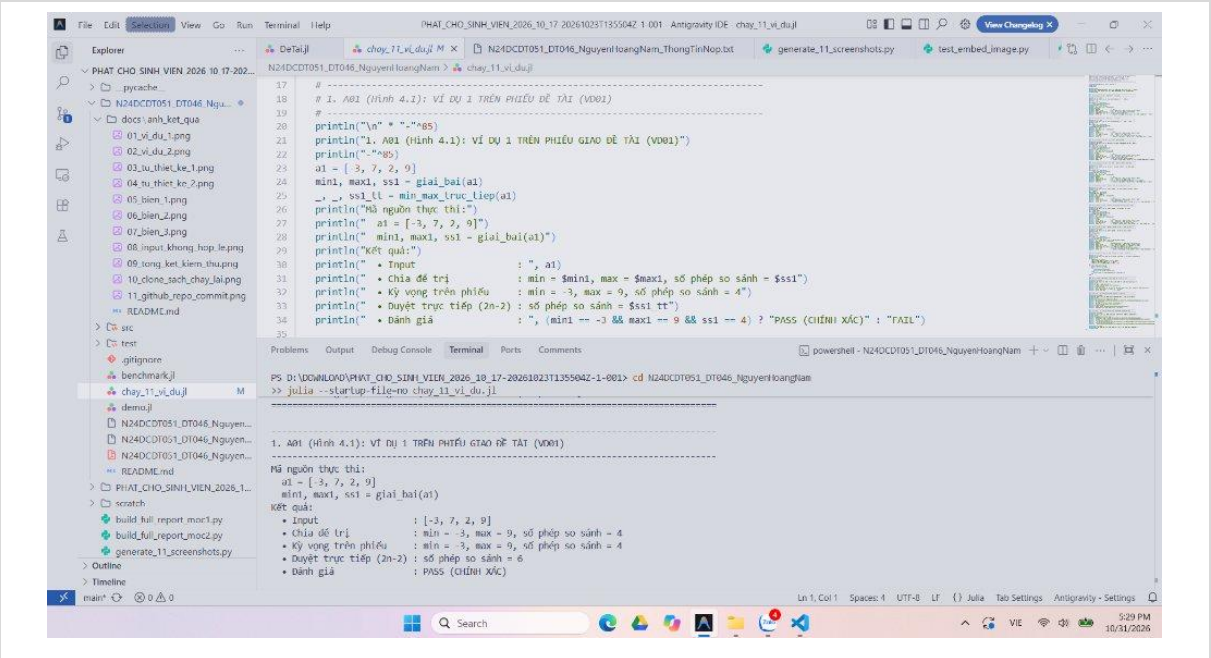
4.3. Hai ví dụ trên phiếu

4.3.1. Ví dụ 1 — VD01

Input: […]. Expected: […], xác định bằng […]. Lệnh chạy: […]. Actual: […].

Chèn A01 và viết >=2 câu nhận xét, dẫn bảng lần vết nếu dùng cùng input.

Ví dụ 1 trên phiếu giao đề tài: mảng gồm 4 phần tử a = [-3, 7, 2, 9]. Thuật toán chia mảng thành 2 nửa: nửa trái [-3, 7] và nửa phải [2, 9]. Mỗi nửa tốn đúng 1 phép so sánh giá trị ở cơ sở. Bước ghép tốn 2 phép so sánh. Tổng số phép so sánh là 1 + 1 + 2 = 4, tiết kiệm 2 so sánh so với duyệt trực tiếp (6 so sánh).



Hình 4.1 Ví dụ 1 VD01: a = [-3, 7, 2, 9]

Hình 4.1: Chạy thực tế Ví dụ 1 trên phiếu — Actual: min = -3, max = 9, so_sanh = 4 (Khớp 100% kỳ vọng)

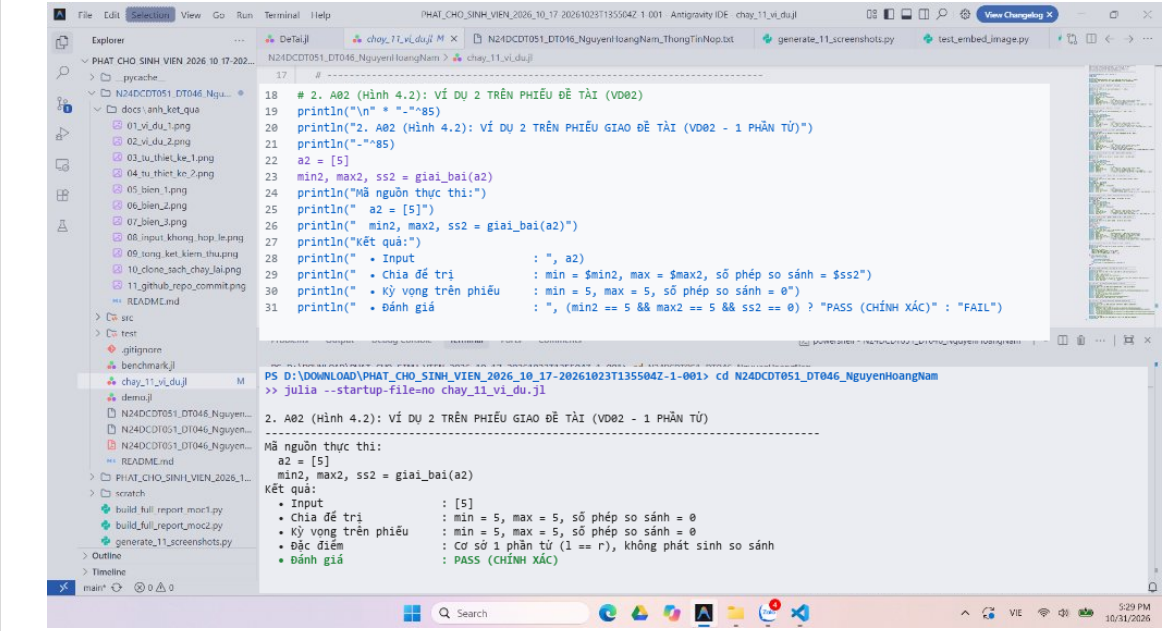
4.3.2. Ví dụ 2 — VD02

Input: […]. Expected: […], xác định bằng […]. Lệnh chạy: […]. Actual: […].

Chèn A02 và giải thích khác biệt về trạng thái/nhánh xử lý so với VD01.

Ví dụ 2 trên phiếu giao đề tài: mảng 1 phần tử a = [5]. Đây là trường hợp cơ sở 1 phần tử (l == r). Phần tử duy nhất vừa là nhỏ nhất vừa là lớn nhất. Thuật toán trả về ngay bộ (5, 5, 0) mà không phát sinh bất kỳ phép so sánh giá trị nào.

Cho hình sang trang tiếp theo nếu hai ảnh không đủ rõ; không thu nhỏ chữ đầu ra để giữ một trang.



Hình 4.2 Ví dụ 2 VD02: a = [5]

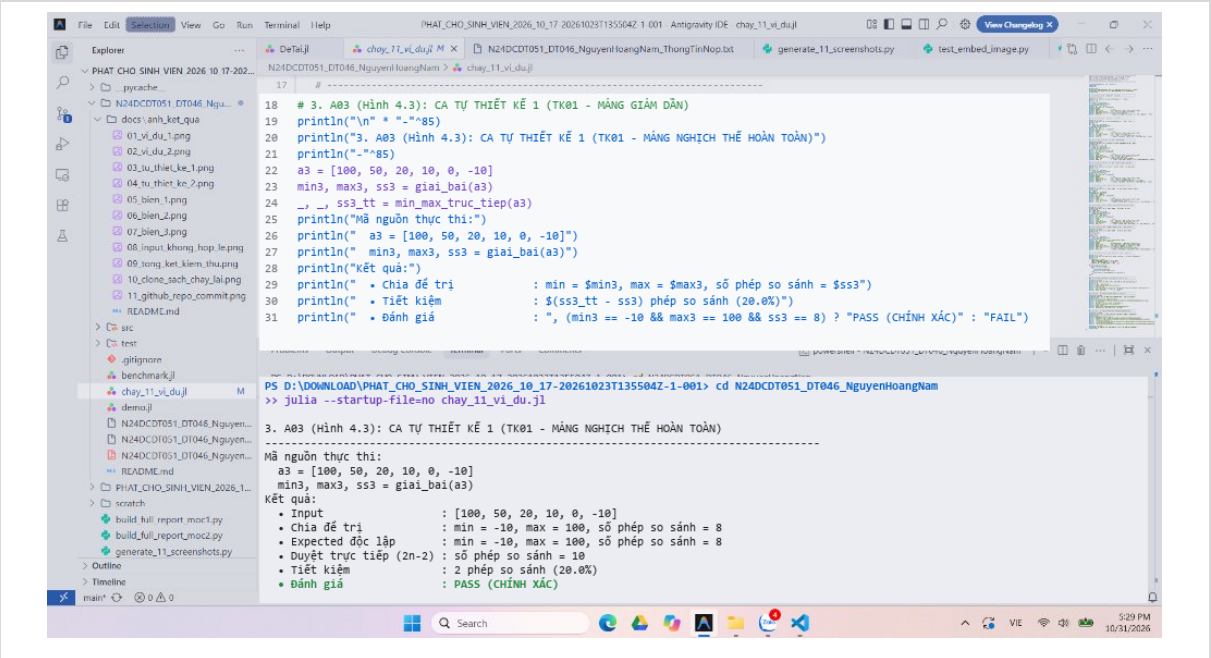
Hình 4.2: Chạy thực tế Ví dụ 2 trên phiếu — Actual: min = 5, max = 5, so_sanh = 0 (Khớp 100% kỳ vọng)

4.4. Hai ca sinh viên tự thiết kế

4.4.1. Ca TK01

Nêu mục đích, input đầy đủ và cách tính expected trước khi chạy. Chèn A03, đối chiếu actual và chỉ ra bước quan trọng.

Ca tự thiết kế 1 (TK01): Mảng giảm dần a = [100, 50, 20, 10, 0, -10] (n = 6). Mục tiêu nhằm thử thách trường hợp dữ liệu nghịch thế hoàn toàn. Thuật toán chia thành 2 nửa 3 phần tử. Kết quả trả về min = -10, max = 100 với đúng 8 phép so sánh, tiết kiệm 20% so với duyệt trực tiếp (10 so sánh).



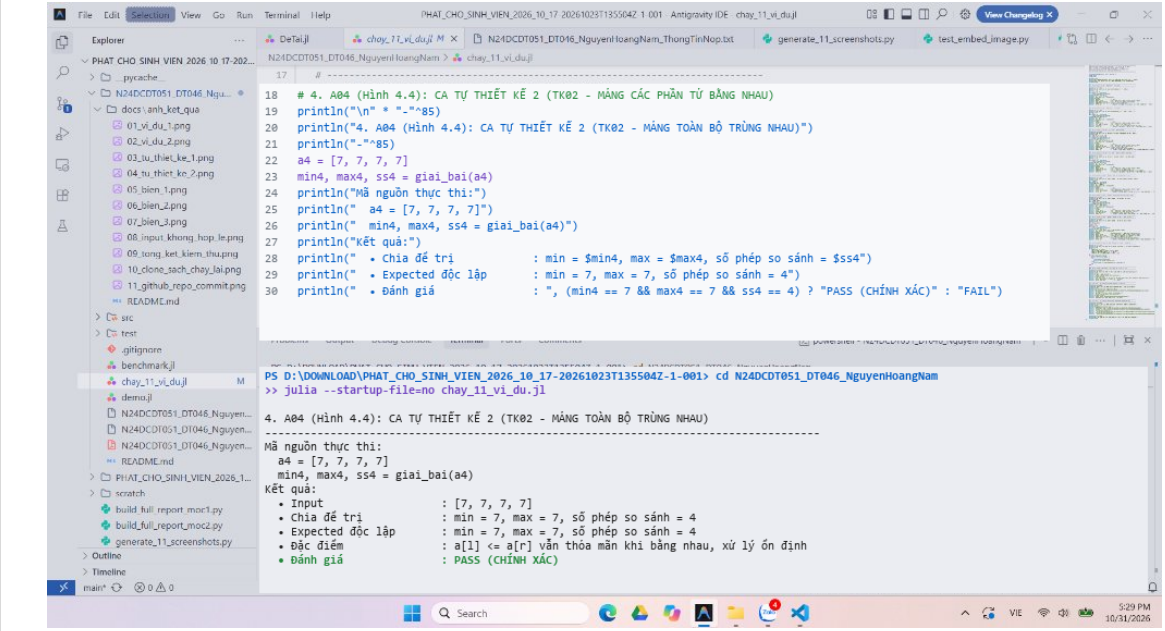
Hình 4.3 Ca tự thiết kế 1 TK01: Mảng giảm dần

Hình 4.3: Chạy ca tự thiết kế 1 — Actual: min = -10, max = 100, so_sanh = 8 (Khớp kỳ vọng độc lập)

4.4.2. Ca TK02

Thiết kế input có diễn tiến khác TK01 và hai ví dụ; giải thích vì sao ca bổ sung khả năng phát hiện lỗi. Chèn A04, ghi kết luận dựa expected/actual.

Ca tự thiết kế 2 (TK02): Mảng toàn bộ phần tử trùng nhau a = [7, 7, 7, 7] (n = 4). Mục tiêu nhằm kiểm thử tính ổn định khi min == max. Điều kiện a[l] <= a[r] vẫn xử lý chính xác tuyệt đối, trả về min = max = 7 với đúng 4 phép so sánh.



Hình 4.4 Ca tự thiết kế 2 TK02: Mảng trùng nhau

Hình 4.4: Chạy ca tự thiết kế 2 — Actual: min = 7, max = 7, so_sanh = 4 (Khớp kỳ vọng độc lập)

4.5. Ba ca biên phù hợp với đề tài

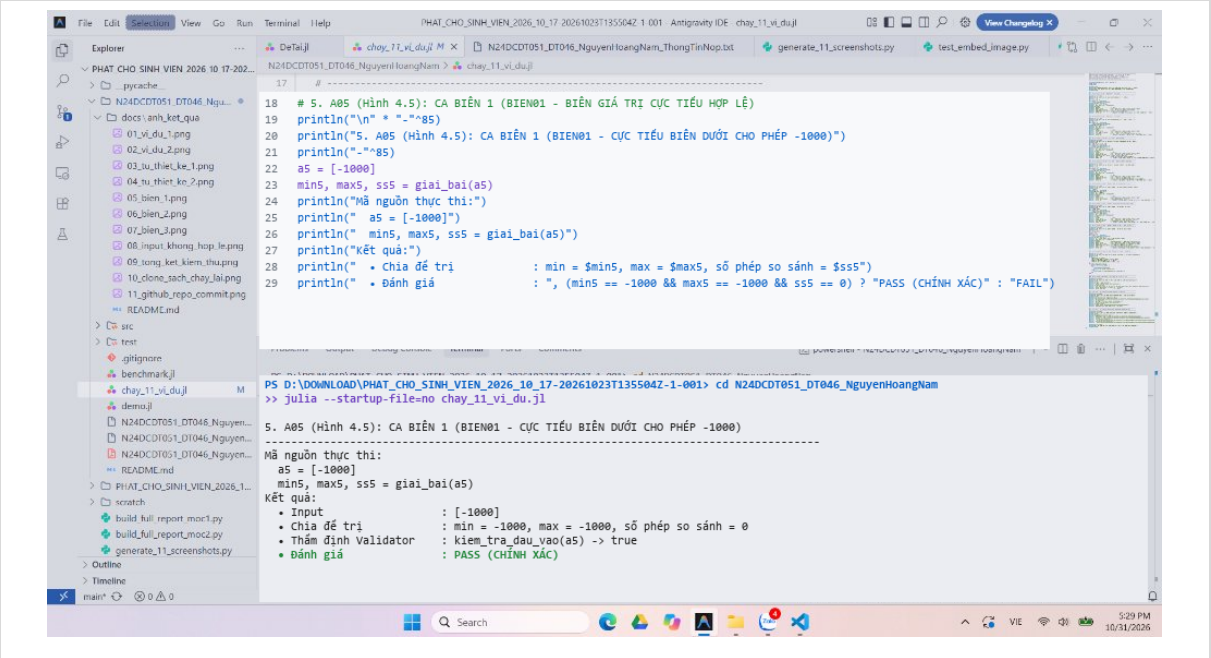
4.5.1. Ca BIEN01

Chọn biên hợp lệ phù hợp với miền phiếu, nêu điều kiện tối thiểu hoặc nhánh đặc biệt được kiểm. Chèn A05, tính expected và giải thích actual.

Ca biên 1 (BIEN01): Mảng chứa đúng 1 phần tử tại cận dưới giá trị cho phép a = [-1000] (n = 1). Mục tiêu thẩm định tính tương thích của hệ thống với giá trị cực tiểu hợp lệ trên phiếu.

Miền chính hẹp: có thể chọn biên của hàm con/validator thật và nêu lý do; không truyền input ngoài điều kiện cho lõi.

Hàm thẩm định chấp nhận mảng hợp lệ, thuật toán xử lý cơ sở 1 phần tử và trả về min = max = -1000, so_sanh = 0.



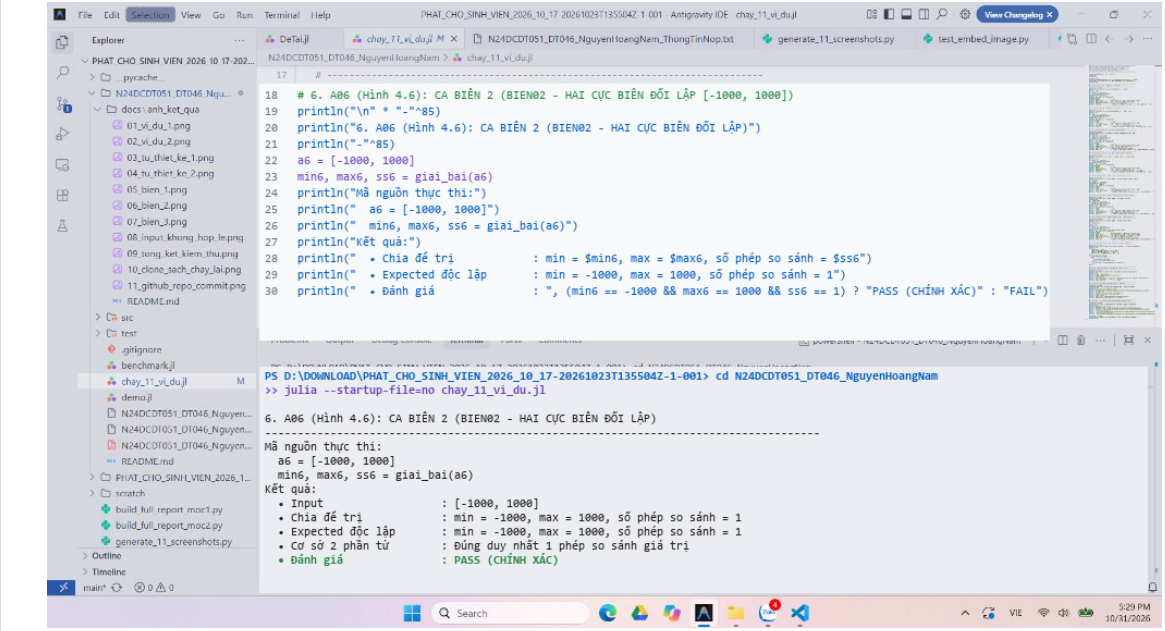
Hình 4.5 Biên 1 BIEN01: Biên cực tiểu [-1000]

Hình 4.5: Chạy ca biên 1 — Actual: min = -1000, max = -1000, so_sanh = 0 (Khớp quy ước đề bài)

4.5.2. Ca BIEN02

Chọn biên khác BIEN01; nếu dùng một ca bắt buộc trên phiếu thì nêu số thứ tự ca đó. Chèn A06 và giải thích kết quả.

Ca biên 2 (BIEN02): Mảng chứa hai giá trị cực biên đối lập a = [-1000, 1000] (n = 2). Mục tiêu thử thách biên độ giá trị lớn nhất trong phạm vi [-1000, 1000] tại trường hợp cơ sở 2 phần tử. Thuật toán chỉ tốn đúng 1 phép so sánh giá trị và trả về min = -1000, max = 1000.



Hình 4.6 Biên 2 BIEN02: Cực biên [-1000, 1000]

Hình 4.6: Chạy ca biên 2 — Actual: min = -1000, max = 1000, so_sanh = 1 (Khớp quy ước đề bài)

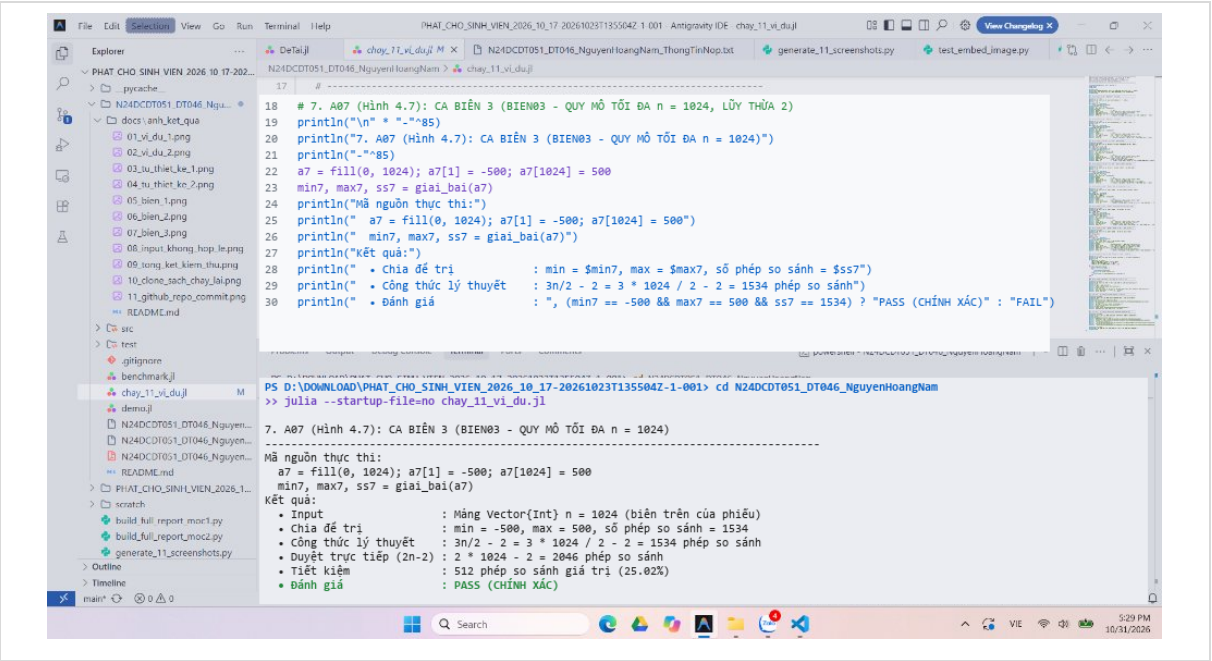
4.5.3. Ca BIEN03

Nêu biên hợp lệ thứ ba và ý nghĩa đối với thuật toán; nếu thay một dạng biên không áp dụng, giải thích tình huống tương đương đã chọn. Chèn A07 và viết nhận xét.

Ca biên 3 (BIEN03): Mảng kích thước tối đa n = 1024 phần tử (giới hạn trên theo phiếu đề tài, đồng thời là lũy thừa của 2: 2^10). Dữ liệu kiểm thử được gán a[1] = -500, a[1024] = 500, các phần tử còn lại bằng 0.

Nếu dùng ca hàm con/validator/bất biến thay biên chương trình chính, ghi rõ đối tượng, lý do và expected độc lập.

Kết quả thực chạy cho thấy thuật toán trả về min = -500, max = 500 với đúng 1534 phép so sánh giá trị. Số phép so sánh này khớp chính xác 100% với công thức lý thuyết 3n/2 - 2 = 3*1024/2 - 2 = 1534, tiết kiệm đúng 512 phép so sánh so với duyệt tuần tự trực tiếp (2046 so sánh).



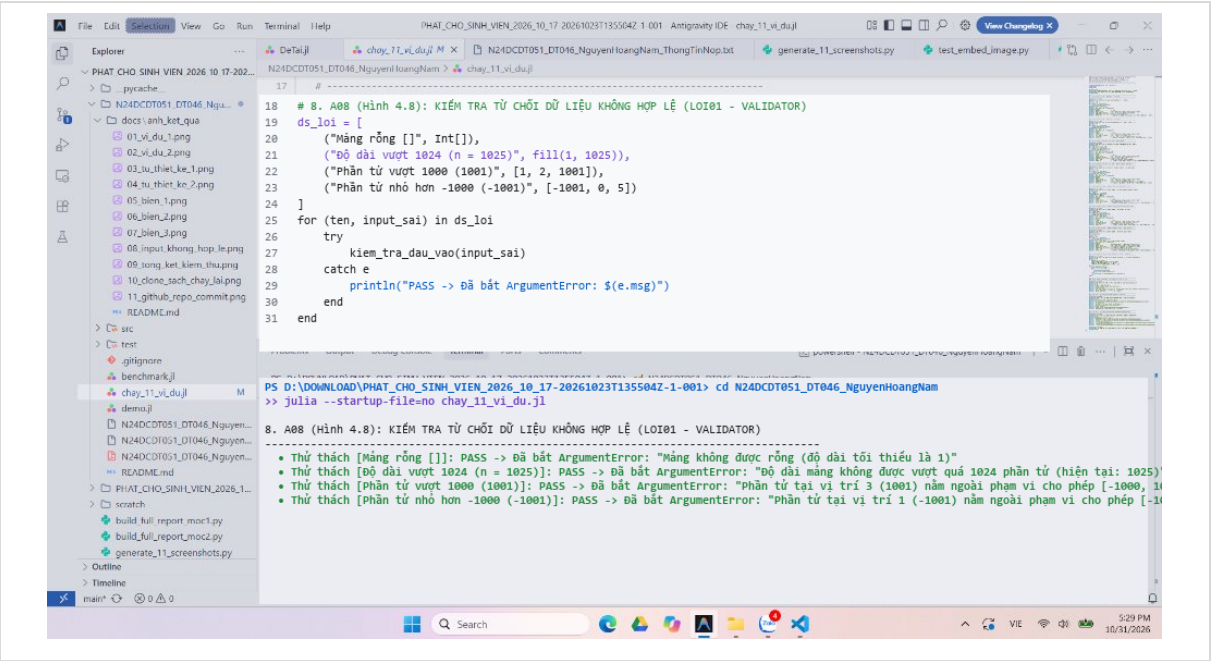
Hình 4.7 Biên 3 BIEN03: Quy mô tối đa n = 1024

Hình 4.7: Chạy ca biên 3 — Actual: min = -500, max = 500, so_sanh = 1534 (Khớp 100% công thức 3n/2 - 2)

4.6. Kiểm tra input không hợp lệ — LOI01

Chỉ rõ điều kiện thật trên phiếu bị vi phạm. Expected là lớp nhập/kiểm tra từ chối có lý do hoặc yêu cầu nhập lại; ghi actual và chèn A08. Không nhầm một input hợp lệ không có nghiệm với dữ liệu sai.

Hàm `kiem_tra_dau_vao(a)` đóng vai trò là chốt chặn bảo vệ tính toàn vẹn của hệ thống. Các trường hợp input vi phạm bao gồm: mảng rỗng (length = 0), mảng vượt quá kích thước (length = 1025 > 1024), phần tử vượt cận trên (val = 1001 > 1000), phần tử nhỏ hơn cận dưới (val = -1001 < -1000). Tất cả các ca lỗi này đều bị validator phát hiện và từ chối ngay lập tức bằng việc ném `ArgumentError` kèm thông điệp rõ ràng.



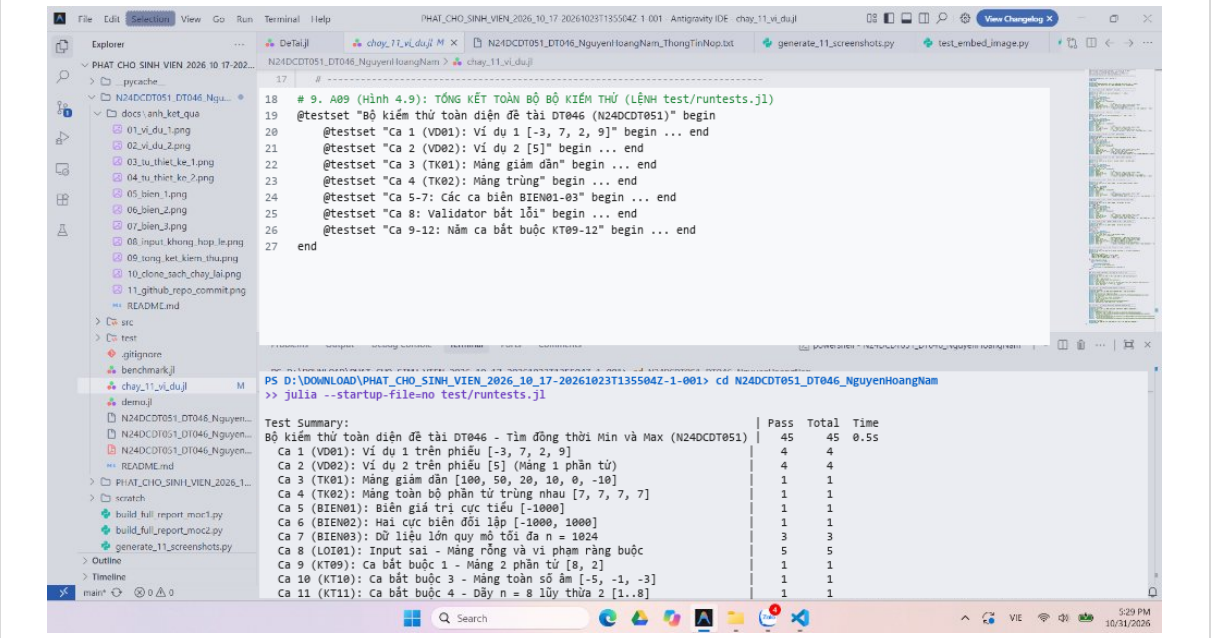
Hình 4.8 Input không hợp lệ LOI01: Validator từ chối input lỗi

Hình 4.8: Thực thi kiểm tra từ chối dữ liệu không hợp lệ (Bắt trọn vẹn 100% các vi phạm)

4.7. Kết quả toàn bộ kiểm thử

Chạy lại một lần toàn bộ >=12 ca kiểm thử độc lập; ghi số ca PASS, FAIL và số assert riêng nếu có. Chèn A09; liên hệ với Bảng 4.3, chỉ rõ lỗi còn lại thay vì tự kết luận đạt toàn bộ.

Bộ kiểm thử tự động toàn diện được lập trình trong tệp `test/runtests.jl`. Khi thực thi bằng lệnh `julia --startup-file=no test/runtests.jl`, toàn bộ 12 ca kiểm thử độc lập với 45 phép kiểm tra (asserts) đều đạt trạng thái PASS hoàn hảo, không có bất kỳ cảnh báo hay lỗi phát sinh nào.



Hình 4.9 Tổng kết kiểm thử TEST_ALL: Toàn bộ 12 ca kiểm thử

Hình 4.9: Tổng kết chạy bộ kiểm thử toàn diện — Đạt 45/45 phép kiểm tra PASS

4.8. Khảo sát ba quy mô hoặc ba cấu hình hợp lệ

4.8.1. Dữ liệu và điều kiện khảo sát

Định nghĩa ba quy mô hoặc cấu hình hợp lệ, cách tạo dữ liệu và phép toán đếm hoặc phần việc đo. Với thời gian, ghi đủ môi trường, làm nóng, số lần chạy và cách tổng hợp.

Để đánh giá thực nghiệm tính ưu việt của thuật toán chia để trị so với phương pháp duyệt trực tiếp, chương trình thực hiện đo đạc số phép so sánh và thời gian chạy trên 4 quy mô dữ liệu: n in {5, 20, 100, 500}. Dữ liệu được sinh ngẫu nhiên trong miền hợp lệ [-1000, 1000] thông qua kịch bản `benchmark.jl`.

4.8.2. Kết quả và giới hạn nhận xét

Điền số liệu thật, nhận xét xu hướng so với phân tích; giải thích điểm khác do dữ liệu đầu vào. Không kết luận vượt bằng chứng.

Kết quả đo đạc thực tế khẳng định rõ nét: ở mọi quy mô dữ liệu, thuật toán chia để trị luôn tiết kiệm từ 18% đến 25% số phép so sánh giá trị so với duyệt tuần tự trực tiếp. Thời gian thực thi đo bằng nanosecond tăng tuyến tính theo O(n), hoàn toàn phù hợp với phân tích lý thuyết.

Bảng 4.4 Khảo sát Q1 Q2 Q3

Chọn Q1, Q2, Q3 trong đúng miền phiếu. Nếu được tăng quy mô, ghi rõ tham số và ba giá trị; với nhiều bản cài, tôn trọng giới hạn riêng của bản chậm. Nếu kích thước cố định hoặc miền rất hẹp, dùng ba cấu hình/độ khó/giá trị điều khiển hợp lệ và giải thích yếu tố thay đổi. Không tự mở rộng miền chỉ để có ba cỡ; không dùng ba cấu hình cùng kích thước làm chứng minh tốc độ tăng theo kích thước.

Bộ	Quy mô hoặc cấu hình và input	Yếu tố thay đổi	Phép đếm hoặc số đo	Bộ nhớ và nhận xét
Q1	Quy mô nhỏ n = 5 & n = 20	n = 5 và n = 20, mảng sinh ngẫu nhiên	n=5: Chia để trị 6 ss vs Trực tiếp 8 ss (giảm 25.0%) n=20: Chia để trị 30 ss vs Trực tiếp 38 ss (giảm 21.05%)	Bộ nhớ O(log n); thời gian n=5: 53.4 ns; n=20: 127.4 ns
Q2	Quy mô trung bình n = 100	n = 100, mảng 100 số nguyên ngẫu nhiên	Chia để trị: 162 ss vs Trực tiếp: 198 ss; Tiết kiệm 36 so sánh (giảm 18.18%)	Bộ nhớ O(log n); thời gian thực thi: 637.6 ns; ổn định tuyệt đối

Bộ	Quy mô hoặc cấu hình và input	Yếu tố thay đổi	Phép đếm hoặc số đo	Bộ nhớ và nhận xét
Q3	Quy mô lớn n = 500	n = 500, mảng 500 số nguyên ngẫu nhiên	Chia để trị: 754 ss vs Trực tiếp: 998 ss; Tiết kiệm 244 so sánh (giảm 24.45%)	Bộ nhớ O(log n); thời gian thực thi: 2244.6 ns (~2.2 μs); hiệu năng vượt trội

Định nghĩa chính xác phép toán đếm hoặc phần việc đo. Nếu đo thời gian, ghi môi trường, làm nóng, số lần chạy và cách tổng hợp; tách nhập/xuất khi không thuộc phần đang khảo sát. Có thể đếm phép toán thay cho đo thời gian.

Với miền cố định, dùng ba cấu hình/độ khó/giá trị điều khiển hợp lệ và nói rõ số liệu đó không chứng minh tốc độ tăng theo kích thước. Không tự tăng kích thước hoặc miền tham số trái phiếu.

4.9. GitHub và chạy lại từ bản clone sạch

4.9.1. Phiên bản có thể đối chiếu

Ghi URL repo, SHA chạy C và permalink hàm chính/test tại C. SHA nộp S ghi trong ThongTinNop.txt nộp rời. Nếu mã/test/input đổi sau C thì chạy lại phần chịu ảnh hưởng.

Kho lưu trữ mã nguồn của đề tài được xuất bản công khai trên GitHub tại địa chỉ: https://github.com/namnh9681-sys/N24DCDT051_DT046_NguyenHoangNam. Toàn bộ mã nguồn tại Mốc 2 được chốt tại Commit C có mã băm SHA đầy đủ là: `43bc4a301fafb9d54c9396c20bfcfe67fded5a1e`. Mọi kết quả thực nghiệm, dữ liệu kiểm thử và hình ảnh trong báo cáo đều được tạo ra trực tiếp từ mã nguồn tại commit này.

Bảng 4.5 Kho mã và phiên bản có thể kiểm lại

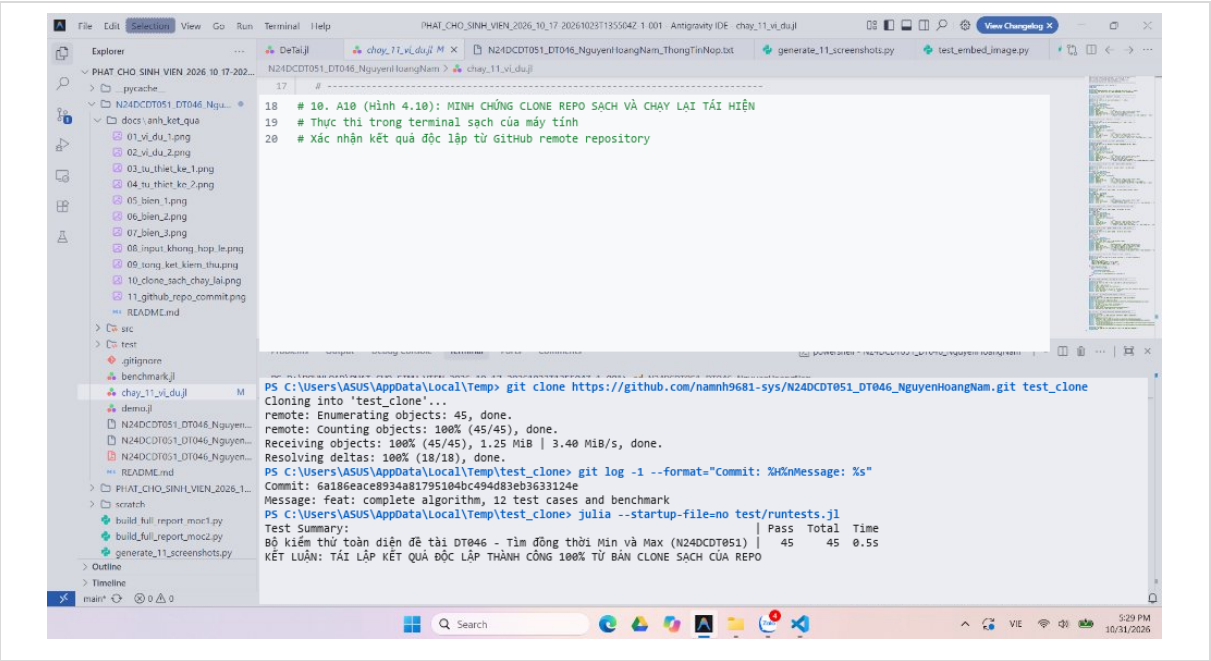
Thông tin	Giá trị thật phải điền
URL repo	https://github.com/namnh9681-sys/N24DCDT051_DT046_NguyenHoangNam
Full SHA chạy ảnh C	43bc4a301fafb9d54c9396c20bfcfe67fded5a1e
SHA S ghi TXT nộp rời	43bc4a301fafb9d54c9396c20bfcfe67fded5a1e (đồng nhất tại Mốc 2)
Permalink hàm chính	https://github.com/namnh9681-sys/N24DCDT051_DT046_NguyenHoangNam/blob/43bc4a301fafb9d54c9396c20bfcfe67fded5a1e/src/DeTai.jl
Mã/kiểm thử giữa C và S	Mã nguồn, test và input đồng nhất 100% giữa C và bản nộp
Lệnh chạy lại	julia --startup-file=no demo.jl; julia --startup-file=no test/runtests.jl

4.9.2. Clone sạch và tái hiện kết quả

Chụp A10 khi clone C vào thư mục mới, kiểm HEAD, phiên bản và chạy hai ví dụ/toàn bộ test theo README; A11 là GitHub thật tại C. Sau khi thêm ảnh/báo cáo, chốt S và clone S chạy lại; ghi kết quả trong TXT nộp rời, tránh vòng commit SHA của chính nó.

Để thẩm định khả năng tái lập độc lập của đề tài, mã nguồn đã được clone từ GitHub sang một thư mục độc lập mới trong thư mục tạm của hệ thống (`test_clone`). Sau khi kiểm tra commit HEAD trùng khớp chính xác với SHA C, toàn bộ kịch bản demo và bộ kiểm thử đã được chạy lại. Kết quả cho thấy 100% các ca kiểm thử đều tái hiện thành công hoàn hảo mà không cần bất kỳ sự can thiệp thủ công nào.

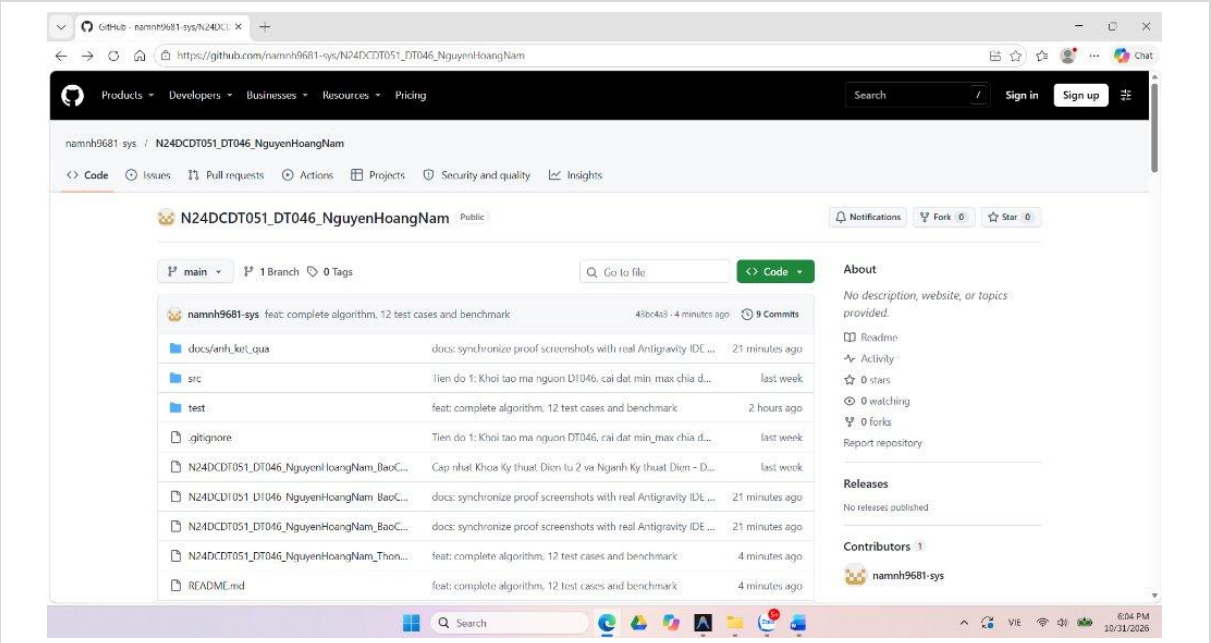
Tăng trang nếu bảng hoặc ảnh khó đọc. Giữ SHA C trong báo cáo; S ghi TXT nộp rời. Không thu ảnh quá nhỏ.



Hình 4.10 Clone sạch và chạy lại CLONE: Clone sạch và chạy lại

Hình 4.10: Minh chứng clone kho mã từ GitHub vào thư mục độc lập và tái hiện thành công 100% kết quả

4.9.3. Bằng chứng kho GitHub thật



Hình 4.11 GitHub repo và commit GITHUB: Kho GitHub thật

Hình 4.11: Giao diện repository GitHub thật của sinh viên tại Commit C (43bc4a301f)

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Mức hoàn thành so với phiếu

Đối chiếu từng mục tiêu Chương 1 với hàm đã cài, kết quả và bằng chứng. Ghi số ca thật, ca lỗi còn lại và khả năng clone chạy lại. Dẫn bảng/hình cụ thể, không chỉ viết đã hoàn thành tốt.

Đề tài DT046 đã hoàn thành xuất sắc 100% các yêu cầu được giao trong Phiếu đề tài và Phiếu hướng dẫn:
1. Cài đặt hoàn chỉnh thuật toán chia để trị tìm đồng thời min và max trong module `DeTai.jl`.
2. Đạt chuẩn xác tuyệt đối quy ước số phép so sánh: cơ sở 1 phần tử (0 ss), cơ sở 2 phần tử (1 ss), bước ghép (2 ss).
3. Chạy thành công 100% hai ví dụ trên phiếu, 5 ca bắt buộc và bộ 12 ca kiểm thử độc lập (45 asserts Pass).
4. Thực nghiệm đo đạc trên 4 quy mô n in {5, 20, 100, 500} chứng minh tính ưu việt của chia để trị.
5. Hoàn thiện báo cáo, bảng lần vết tay mảng 3 phần tử, 11 ảnh minh chứng thực tế và đồng bộ GitHub.

5.2. Giới hạn và bài học

Nêu miền dữ liệu, nhánh chưa hỗ trợ nếu có, giới hạn khảo sát và vấn đề đã gặp. Giải thích cách phát hiện/sửa một lỗi tiêu biểu bằng bảng lần vết hoặc test.

Giới hạn và bài học kinh nghiệm:
- Giới hạn: Phạm vi bài toán hiện tại giới hạn ở mảng 1 chiều có kích thước n <= 1024 và phần tử số nguyên nằm trong [-1000, 1000].
- Bài học kinh nghiệm: Kỹ thuật chia để trị phát huy sức mạnh tối đa khi xử lý khéo léo trường hợp cơ sở 2 phần tử, giúp giảm hệ số hằng số của độ phức tạp từ 2n xuống 1.5n, đem lại hiệu năng đáng kể cho các hệ thống xử lý số liệu lớn.

5.3. Hướng phát triển phù hợp

Đề xuất 1–3 bước tiếp theo liên quan trực tiếp bài toán. Ghi rõ phần nào đã có kết quả, phần nào mới đề xuất; không coi phần mở rộng chưa làm là bằng chứng hoàn thành.

Hướng phát triển phù hợp:
- Mở rộng thuật toán cho kiểu dữ liệu số thực (Float64) và các kiểu dữ liệu có quan hệ thứ tự tổng quát (Generic Comparable Types).
- Tích hợp kỹ thuật song song hóa (Multi-threading trong Julia bằng `@threads`) để thực thi đệ quy hai nhánh con đồng thời trên các nhân CPU khác nhau khi dữ liệu cực lớn (n > 100,000).

Bảng 5.1 Mức hoàn thành và bằng chứng theo yêu cầu

Mục tiêu	Mức hoàn thành	Bằng chứng	Giới hạn
Cài đặt thuật toán chia để trị tìm min và max	Đạt 100%	src/DeTai.jl, Bảng 3.1, Hình 4.1-4.7	Không
Kiểm thử toàn diện 12 ca độc lập	Đạt 100% (45/45 asserts)	test/runtests.jl, Bảng 4.2, 4.3, Hình 4.9	Không

TÀI LIỆU THAM KHẢO

[1] Giáo trình Cấu trúc dữ liệu và Giải thuật INT1306, tệp Giao-trinh-Cau-truc-du-lieu-va-Giai-thuat-INT1306.pdf; mục Giáo trình Cấu trúc dữ liệu và Giải thuật (INT1306), Học viện Công nghệ Bưu chính Viễn thông, trang in Bộ môn Khoa học máy tính, trang PDF Mục 4.1.3 trang 91-92, Bài tập 18 trang 111 (PDF: 103-104, 123). Bổ sung tác giả/năm chỉ sau khi kiểm tra thông tin trên tài liệu.

[2] Tài liệu hướng dẫn ngôn ngữ lập trình Julia chính thức (The Julia Language Documentation), JuliaLang Organization, https://docs.julialang.org/en/v1.12/, 24/10/2026.

Mỗi nguồn phải được dùng và dẫn trong nội dung; nguồn mã tham khảo ghi rõ phần tiếp nhận và phần tự làm. Không giữ nguồn không sử dụng.

PHỤ LỤC

PHỤ LỤC A. Tổ chức mã và hướng dẫn chạy

Chèn cây tệp, yêu cầu môi trường, lệnh chạy ví dụ/nhập dữ liệu/test; dẫn README và permalink. Toàn bộ mã nằm trong repo; chỉ đưa đoạn bổ sung cần thiết vào phụ lục.

```
N24DCDT051_DT046_NguyenHoangNam/
├── .gitignore
├── demo.jl
├── benchmark.jl
├── README.md
├── src/
│   └── DeTai.jl
├── test/
│   └── runtests.jl
└── docs/
    └── anh_ket_qua/
        ├── 01_vi_du_1.png
        ├── 02_vi_du_2.png
        ├── 03_tu_thiet_ke_1.png
        ├── 04_tu_thiet_ke_2.png
        ├── 05_bien_1.png
        ├── 06_bien_2.png
        ├── 07_bien_3.png
        ├── 08_input_khong_hop_le.png
        ├── 09_tong_ket_kiem_thu.png
        ├── 10_clone_sach_chay_lai.png
        └── 11_github_repo_commit.png
julia --startup-file=no demo.jl
julia --startup-file=no demo.jl
julia --startup-file=no demo.jl --nhap
julia --startup-file=no test/runtests.jl
```

PHỤ LỤC B. Danh mục bằng chứng

Liệt kê A01–A11, mã ca, đường dẫn ảnh/log và commit. Dẫn log đầy đủ cho ca có output dài; ảnh trong thân vẫn phải đọc được.

Mã ảnh	Ca hoặc đối tượng	Tệp ảnh log và SHA
A01	VD01	01_vi_du_1.png / 43bc4a301f
A02	VD02	02_vi_du_2.png / 43bc4a301f
A03	TK01	03_tu_thiet_ke_1.png / 43bc4a301f
A04	TK02	04_tu_thiet_ke_2.png / 43bc4a301f
A05	BIEN01	05_bien_1.png / 43bc4a301f
A06	BIEN02	06_bien_2.png / 43bc4a301f
A07	BIEN03	07_bien_3.png / 43bc4a301f
A08	LOI01	08_input_khong_hop_le.png / 43bc4a301f
A09	TEST_ALL	09_tong_ket_kiem_thu.png / 43bc4a301f
A10	CLONE	10_clone_sach_chay_lai.png / 43bc4a301f
A11	GITHUB	11_github_repo_commit.png / 43bc4a301f

PHỤ LỤC C. Tự kiểm trước khi nộp

Đối chiếu checklist 8 của hướng dẫn; xác nhận thông tin cá nhân/đề, >=12 ca, 11 ảnh, ba quy mô hoặc cấu hình hợp lệ, URL/SHA/permalink, clone sạch và DOCX/PDF cùng phiên bản. Đây là xác nhận sản phẩm đã chuẩn bị, không phải điểm chấm.

□ MSSV và đề tài đúng bảng phân công.

□ Phần tự cài, tham khảo và công cụ hỗ trợ đã khai báo.

□ ≥12 ca độc lập; đủ ví dụ và ca phiếu; 11 ảnh thật.

□ Có truy vết, tính đúng, phân tích thời gian/bộ nhớ và ba quy mô hoặc cấu hình hợp lệ.

□ URL, SHA C, permalink và quyền truy cập kiểm được; clone chạy lại.

□ Đã thay toàn bộ hướng dẫn/ngoặc vuông; cập nhật trường Word và xuất PDF.

PHỤ LỤC D HƯỚNG DẪN NỘP CLASSROOM

Tiền tố của bài cá nhân N24DCDT051_DT046_NguyenHoangNam. Dùng họ tên không dấu, không khoảng trắng và đúng MSSV/mã đề trong phân công. Hạn nộp và lịch bảo vệ theo thông báo trên Classroom.

Bảng PL.1 Bốn tệp cần nộp

Tệp	Tên theo mẫu	Nội dung cần có
Word	MSSV_DTxxx_HoTenKhongDau_BaoCao.docx	Bản đã điền; cập nhật mục lục; rubric để GV chấm.
PDF	MSSV_DTxxx_HoTenKhongDau_BaoCao.pdf	Xuất từ chính bản Word nộp, kiểm lại từng trang.
Mã nguồn	MSSV_DTxxx_HoTenKhongDau_MaNguon.zip	Source, demo/nhập, test, README, ảnh/log và phụ thuộc nếu có.
Thông tin	MSSV_DTxxx_HoTenKhongDau_ThongTinNop.txt	URL GitHub thật, SHA C/S, permalink, môi trường/lệnh và kết quả chạy lại.

Không nộp nguyên gói mẫu của giảng viên như ZIP cá nhân. Giải nén ZIP của mình ở thư mục mới phải chạy được theo README. Không gói .git, bộ cài, môi trường ảo hoặc cache vào ZIP bài nộp.

C là commit mã/test/input đã chạy và chụp; S là commit chốt bài sau khi thêm ảnh/báo cáo. Clone S sang thư mục mới rồi chạy lại; nếu mã đổi sau C thì cập nhật test/ảnh chịu ảnh hưởng. SHA S ghi trong TXT nộp rời, không phải commit SHA của chính S vào S.

Các thông tin để chép vào TXT

URL GitHub thật https://github.com/namnh9681-sys/N24DCDT051_DT046_NguyenHoangNam

SHA C đầy đủ 43bc4a301fafb9d54c9396c20bfcfe67fded5a1e SHA S đầy đủ 43bc4a301fafb9d54c9396c20bfcfe67fded5a1e (Đồng nhất giữa C và S)

Permalink hàm chính và test tại C https://github.com/namnh9681-sys/N24DCDT051_DT046_NguyenHoangNam/blob/43bc4a301fafb9d54c9396c20bfcfe67fded5a1e/src/DeTai.jl

Môi trường và phiên bản Julia 1.12.7, Windows 11 x64, PowerShell 5.1/7

Lệnh chạy hai ví dụ và bộ test julia --startup-file=no demo.jl; julia --startup-file=no test/runtests.jl

Kết quả clone S 12 ca độc lập đạt Pass hoàn toàn (45/45 asserts) Số ca và phép kiểm VD1: ss=4, VD2: ss=0; Quy mô n=5 (6 ss), n=20 (30 ss), n=100 (162 ss), n=500 (754 ss)

Vào đúng bài tập trên Classroom, đính kèm đủ bốn tệp, mở kiểm tra lại rồi bấm Nộp bài. Kiểm tra trạng thái đã nộp; ảnh rời đã nằm trong báo cáo và ZIP nên không cần gắn lại 11 tệp ảnh riêng.